248. Hatch, B., and J. Lee, *Hacking Linux Exposed,* McGraw-Hill: Osborne, 2003, pp. 384-386.

249. Toxen, B., *Real World Linux Security,* 2nd ed., Prentice Hall PTR, 2002.

250. "Modules/Applications Available or in Progress," modified May 31, 2003, <www.kernel.org/pub/linux/libs/pam/modules.html>.

251. Morgan, A., "The Linux-PAM Module Writers' Guide," May 9, 2002, <www.kernel.org/pub/linux/libs/pam/Linux-PAM-html/pam_modules.html>.

252. Hatch, B., and J. Lee, *Hacking Linux Exposed,* McGraw-Hill: Osborne, 2003, pp. 15-19.

253. Linux man pages, "CHATTR(l), change file attributes on a Linux second extended file system," <nodevice.com/cgi-bin/searchman?topic=chattr>.

254. Hatch, B., and J. Lee, *Hacking Linux Exposed,* McGraw-Hill: Osborne, 2003, p. 24.

255. Smalley, S.; T. Fraser; and C. Vance, "Linux Security Modules: General Security Hooks for Linux," <lsm.immunix.org/docs/overview/linuxsecuritymodule.html>.

256. Jaeger, T.; D. Safford; and H. Franke, "Security Requirements for the Deployment of the Linux Kernel in Enterprise Systems," <oss.software.ibm.com/linux/papers/security/les_whitepaper.pdf>.

257. Wheeler, D., "Secure Programming for Linux HOWTO," February 9, 2000, <www.theorygroup.com/Theory/FAQ/Secure-Programs-HOWTO.html>.

258. Wright, C, et al., "Linux Security Modules: General Security Support for the Linux Kernel," 2002, <lsm.immun-ix.org/docs/lsm-usenix-2002/html/>.

259. Bryson, D., "The Linux CryptoAPI: A User's Perspective," May 31,2002, <www.kerneli.org/howto/index.php>.

260. Bryson, D., "Using CryptoAPI," May 31, 2002, <www.kerneli.org/howto/node3.php>.

261. Bovet, D., and M. Cesati, *Understanding the Linux Kernel,* O'Reilly, 2001.

262. Rubini, A., and J. Corbet, *Linux Device Drivers,* O'Reilly, 2001.

But, soft! what light through yonder window breaks?
It is the east, and Juliet is the sun!

—William Shakespeare—

An actor entering through the door, you've got nothing. But if he enters through the window, you've got a situation.

-Billy Wilder-

Chapter 21

Case Study: Windms XP

Objectives

After reading this chapter, you should understand:

- *the history of DOS and Windows operating systems.*
- *the Windows XP architecture.*
- *the various Windows XP subsystems.*
- *asynchronous and deferred procedure calls.*
- *how user processes, the executive and the kernel interact.*
- *how Windows XP performs process, thread, memory and file management.*
- *the Windows XP I/O subsystem.*
- *how Windows XP performs interprocess communication.*
- *networking and multiprocessing in Windows XP.*
- *the Windows XP security model.*

Chapter Outline

Web Resources | Key Terms | Exercises | Recommended Reading | Works Cited

21.1 Introduction

Windows XP, released by the Microsoft Corporation in 2001, combines Microsoft's corporate and consumer operating system lines. By early 2003, over one-third of all Internet users ran Windows XP, making it the most widely used operating system.[1]

Windows XP ships in five editions. *Windows XP Home Edition* is the desktop edition, and the other editions provide additional features. *Windows XP Professional* includes extra security and privacy features, more data recovery support and broader networking capabilities. *Windows XP Tablet PC Edition* is built for notebooks and laptops that need enhanced support for wireless networking and digital pens. *Windows XP Media Center Edition* provides enhanced multimedia support.[2] *Windows XP 64-Bit Edition* is designed for applications that manipulate large amounts of data, such as programs that perform scientific computing or render 3D graphics. At the time of this writing, Microsoft plans to release a sixth edition, *Windows XP 64-Bit Edition for 64-Bit Extended Systems,* which is designed specifically to support AMD Opteron and Athlon 64 processors.[3] Windows XP 64-Bit Edition currently executes on Intel Itanium II processors.

Despite their differences, all Windows XP editions are built on the same core architecture, and the following case study applies to all editions (except when we explicitly differentiate between editions). This case study investigates how a popular operating system implements the components and strategies we have discussed throughout the book.

21.2 History

In 1975, a Harvard University junior and a young programmer at Honeywell showed up at Micro Instrumentation and Telemetry Systems (MITS) headquarters with a BASIC compiler. The two men had written the compiler in just eight weeks and had never tested it on the computer for which it was intended. It worked on the first run. This auspicious beginning inspired the student, Bill Gates, and the programmer, Paul Allen, to drop everything they were doing and move to Albuquerque, New Mexico, to found Microsoft (see the Biographical Note, Bill Gates).[4]-[5] By 2003, the two-man company had become a global corporation, employing over 50,000 people and grossing a yearly revenue of over $28 billion.[6] Microsoft is the second most valuable and the sixth most profitable company in the world. *Forbes Magazine* in 2003 listed Bill Gates as the richest man on Earth, worth over $40.7 billion, and Paul Allen as the fourth richest man, worth $20.1 billion.[7]

Microsoft released the Microsoft Disk Operating System—MS-DOS 1.0—in 1981. MS-DOS 1.0 was a 16-bit operating system that supported 1MB of main memory (1MB was enormous by the standards of the time). The system ran one process at a time in response to user input from a command line. All programs executed in **real mode,** which provided them direct access to all of main memory including the portion storing the operating system.[8] MS-DOS 2.0, released two years later, supported a 10MB hard drive and 360KB floppy disks.[9] Subsequent

Microsoft operating systems continued the trend of increasing disk space and supporting an ever-growing number of peripheral devices.

Microsoft released Windows 1.0, its graphical-user-interface (GUI)-based operating system, in 1985. The GUI foundation had been developed by Xerox in the 1970s and popularized by Apple's Macintosh computers. The Windows GUI

Biographical Note

Bill Gates

William H. Gates III was born in Seattle, Washington, in 1955.[10] His first experience with computers was in high school.[11,12] He befriended Paul Allen, a fellow computer enthusiast, and with two other students formed the Lakeside Programming Group. The group obtained computer time from nearby companies by arranging deals to find bugs in the companies' systems. They also earned money by forming a small and short-lived company named Traf-O-Data, writing software to process traffic flow data.[13, 14]

Gates went on to Harvard University, but in 1974 Intel announced the 8080 chip, and Gates and Paul Allen realized that the future of computing was microprocessing.[15, 16, 17] When the MITS Altair 8800 was announced only months later—the world's first microcomputer—Gates and Allen quickly contacted MITS to say they had a implementation of BASIC (Beginner's All-purpose Symbolic Instruction Code) for the Altair.[18, 19] In fact they had no such thing, but Altair was interested; Gates took a leave of absence from Harvard (he never returned) and he and Allen ported BASIC to the Altair in just over one month, based only on Intel's manual for the 8080.[20, 21, 22] Early personal computers did not have operating systems, so all that was needed was a language interpreter built for the specific system to run programs. Gates' and Allen's implementation of BASIC was, in effect, the first operating system for the Altair. It was also the beginning of Microsoft.[23, 24]

Over the next several years, Microsoft continued to develop their BASIC language and licensed it to other new microcomputer companies including Apple and Commodore.[25] Microsoft also implemented other languages for microcomputers and, because they had a jump-start on other microcomputer software companies, began to take over the field.[26] In 1980, Gates hired his Harvard classmate, Steve Ballmer, to help with the business side of the growing company (Ballmer is the current CEO of Microsoft.)[27, 28] Only two years later, Microsoft developed MS-DOS based on Seattle Computing Product's 86-DOS; it was offered as the cheapest operating system option for IBM's new line of personal computers and was a great success. In 1984 they broke into the office applications market with Word and Excel for the Apple Macintosh. The following year they released Windows 1.0, one of the first GUI operating systems for IBM-compatibles.[29] Microsoft's commercial success has continued to mount with progressive versions of Windows and their Microsoft Office software. Gates is currently the Chairman and Chief Software Architect of Microsoft.[30] He is also the richest person in the world.[31]

improved with each version. The window frames in Windows 1.0 could not overlap; Microsoft fixed this problem in Windows 2.0. The most important feature in Windows 2.0 was its support for **protected mode** for DOS programs. Although protected mode was slower than real mode, it prevented a program from overwriting the memory space of another program, including the operating system, which improved system stability. Even in protected mode, however, programs addressed memory directly. In 1990, Microsoft released Windows 3.0, quickly followed by Windows 3.1. This version of the operating system completely eliminated the unsafe real mode. Windows 3.1 introduced an enhanced mode, which allowed Windows applications to use more memory than DOS programs. Microsoft also released Windows for Workgroups 3.1, an upgrade to Windows 3.1 that included network support, especially for local area networks (LANs), which, at the time, were starting to become popular. Microsoft attempted to increase system stability in Windows by tightening parameter checking during API system calls; these changes broke many older programs that exploited "hidden" features in the API.[32, 33]

Emboldened by the success of its home-user-oriented operating systems. Microsoft ventured into the business market. Microsoft hired Dave Cutler, an experienced operating systems designer who helped develop Digital Equipment Corporation's VMS operating system, to create an operating system specifically for the business environment (see the Biographical Note, David Cutler). Developing the new operating system took five years, but in 1993, Microsoft released Windows NT 3.1, a New Technology operating system. In so doing, Microsoft created its corporate line of operating systems, which was built from a separate code base than its consumer line. The two lines were not to merge until the release of Windows XP in 2001, although operating systems from one line used features and ideas from the other. For example, Windows NT 3.1 and Windows 95 included different implementations of the same API. In addition, the business and consumer GUI interfaces were always similar, although the business version tended to lag behind.[34]

Because Windows NT 3.1 was developed for business users, its chief focus was security and stability. Windows NT introduced the New Technology File System (NTFS), which is more secure and more efficient than the then-popular Windows 3.1 FAT and IBM OS/2 HPFS file systems (see the Mini Case Study, OS/2). In addition, the 32-bit operating system protects its memory from direct access by user applications. The NT kernel runs in its own protected memory space. This extra layer of security came at a price; many existing graphics-intensive games could not run on Windows NT because they need to access memory directly to optimize performance. Windows NT 3.1 also lacks much of the multimedia support that made Windows 3.1 popular with home users.[35]

Microsoft released Windows NT 4.0 in 1996. The upgraded operating system provides additional security and networking support and includes the popular Windows 95 user-interface.[36] Windows 2000 (whose kernel is NT 5.0), released in 2000. introduced the Active Directory. This database of users, computers and devices makes it easy for users to find information about resources spread across a network.

In addition, system administrators can configure user accounts and computer settings remotely. Windows 2000 supports secure Kerberos single sign-on authentication and authorization (see Section 19.3.3, Kerberos). In general, Windows 2000 is more secure and has better networking support than previous Windows operating systems. It is Microsoft's last purely business-oriented desktop operating system.[37]

Microsoft continued developing consumer operating systems. It released Windows 95 in 1995 to replace Windows 3.1 for home users. Like Windows NT 3.1, Windows 95 makes the jump to 32-bit addressing, although it supports 16-bit applications for backward compatibility. Windows 95 permits applications to access

Biographical Note

David Cutler

David Cutler graduated from Olivet College in Michigan in 1965[38] with a degree in Mathematics.[39] He first became interested in computers when he worked on a software simulation project at DuPont.[40, 41] Cutler moved to its engineering department, where he was involved in various programming projects, became interested in operating systems and worked on the development of several small, real-time operating systems.[42]

In 1971 Cutler moved to Digital Equipment Corporation (DEC),[43, 44] where he led the team developing the RSX-11M operating system. This operating system was difficult to design for two reasons—there were severe memory limitations due to the PDP-11 's 16-bit addressing, and it had to be compatible with all of the popular PDP-11 models. A few years later. DEC began work on the VAX 32-bit architecture, because increasing application sizes made the memory space accessible by a 16-bit address insufficient.[45]

Cutler managed what was probably DEC'S most important OS project ever—**VMS** for the VAX systems.[46, 47] Because the PDP-11 was so widely used, VMS needed to be backward compatible with the PDP-11 system and it had to be scalable to VAX systems of varying power.[48, 49] Cutler also led the MicroVAX project, DEC's microprocessor version of the VAX.[50]

In 1988, Cutler accepted an offer from Bill Gates to be one of the leaders on Microsoft's successor to OS/2—OS/2 New Technology, or OS/2 NT, which would use the OS/2 API.[51, 52] cutler insisted on bringing 20 of his developers from DEC to form the core of this new team. Meanwhile, Microsoft released Windows 3.0, and its popularity led to a change in plans for Cutler's project. His operating system would now be Windows NT and would primarily support the Win32 interface. However, it needed to support parts of the DOS, POSIX and OS/2 APIs in addition to running Windows 3.0 programs.[53] Inspired by the Mach microkernel operating system. Cutler's team designed Windows NT to have a fairly small kernel with layers of separate modules to handle the various interfaces. Windows NT, originally intended for Microsoft's professional line of operating systems, eventually was used for the consumer line in version 5.1, more popularly known as Windows XP. Cutler remains a lead operating systems architect at Microsoft.

main memory directly. However, Microsoft introduced DirectX, which simulates direct hardware access while providing a layer of protection. Microsoft hoped that DirectX, combined with faster hardware, would give game developers a viable alternative to accessing the hardware directly. Windows 95 introduced multithreading, an improvement on the multitasking support provided by Windows 3.1. Microsoft released Windows 98 in 1998, although most of the changes were minor (primarily increased multimedia and networking support). The biggest innovation was combining Internet Explorer (IE) with the operating system. Bundling IE made Windows easier to use when accessing the Internet and the Web. Windows Millennium Edition (Windows Me), released in 2000, is the last version in the consumer line. Windows Me has even more device drivers and multimedia and networking support than previous versions of Windows. It is also the first consumer version of Windows that cannot boot in DOS mode.[54]

Mini Case Study

OS/2

OS/2 was first released in **1987.** The operating system was a joint project between Microsoft and **IBM** designed to run on IBM's new **PS/2** line of personal computers.[55] OS/2 was based on **DOS,** the most popular operating system at the time of OS/2's release, and was similar to DOS in many ways (e.g., they had similar commands and command prompts). OS/2 also made significant improvements over DOS. OS/2 was the first PC operating system to allow multitasking (however, only one application could be on the screen at a time). OS/2 also supported multithreading, interprocess communication (IPC), dynamic linking and many other features that were not supported by DOS.[56]

OS/2 1.1, released in 1988, was its first version with a graphical user interface. The interface was similar to early versions of Windows. However, unlike Windows and many of today's interfaces, the coordinate (0,0) referred to the lower-left corner of the screen as opposed to the upper-left comer.[57] This made transferring programs from OS/2 to other operating systems difficult.

OS/2 version 2.0, released in 1992, was the first 32-bit operating system for personal computers. OS/2 2 contained the Integrating Platform, designed to run OS/2 2, OS/2 1, DOS and Windows applications. This benefitted OS/2, because there were few applications written for OS/2 2 at the time. As a result, many developers wrote programs specifically for Windows, since they would work on both platforms.[58]

IBM and Microsoft disagreed on the progress and direction of OS/2 because Microsoft wanted to devote more resources to Windows than to OS/2, whereas IBM wished to further develop OS/2 1. In 1990, IBM and Microsoft agreed to work independently On upcoming OS/2 projects: IBM took control of all future versions of OS/2 1 and OS/2 2, while Microsoft developed OS/2 3. Soon after this agreement, Microsoft renamed OS/2 3 to Windows NT.[59]

Microsoft released Windows XP in 2001 (whose kernel is NT 5.1). Windows XP unified the consumer and commercial code bases. Windows XP comes in several editions, each of which is built from the same code base but is tailored to fit the needs of a specific group of users. Every edition of Windows XP combines the stability, security and network support of the corporate line with the multimedia and device support of the consumer line.

21.3 Design Goals

Microsoft's main design goal for Windows XP was reintegrating the code bases from its consumer and business operating system lines. In addition, Microsoft attempted to better meet the traditional goals of each individual line. For example, the corporate line has often focused on providing security and stability, whereas the consumer line provides enhanced multimedia support for games.

When Microsoft created the NT line for business users, its chief focuses were stability, security and scalability.[60] Businesses often employ computers to execute critical applications and store essential information. It is important for the system to be reliable and run continuously without error. The internal architecture described in this case study was developed over many years to enhance the stability of Microsoft operating systems.

Corporate computers need to protect sensitive data while being connected to multiple networks and the Internet. For this reason, Microsoft devoted a great deal of effort to developing an all-encompassing security system. Windows XP uses modern security technologies such as Kerberos, access control lists and packet filtering firewalls to protect the system. The Windows XP security system is built on the codebase of Windows 2000, to which the government gave C2 level certification (the C2 certification applies only to Windows 2000 machines unconnected to a network).[61] At the time of this writing, Microsoft is in the process of gaining the same certification for Windows XP Professional.

Computers must be scalable to meet the needs of different users. Microsoft developed Windows XP 64-Bit Edition to support high-performance desktop computers and Windows XP Embedded to support devices such as routers. Windows XP Professional supports symmetric multiprocessing (see Chapter 15, Multiprocessor Management).

Windows XP incorporates many of the features that made the consumer line popular. The system contains an improved GUI and more multimedia support than previous versions of Windows. For the first time, both business and home users can take advantage of these innovations.

Besides incorporating the strengths of its two lines of operating systems, Microsoft added another goal specifically for Windows XP. Customers had often complained that Windows started too slowly. As a result, Microsoft aimed at dramatically decreasing boot time. Windows XP must be usable in 30 seconds after a user turns on the power, 20 seconds after waking from hibernation (on a laptop) and 5 seconds

after resuming from standby. Later in the chapter, we describe how Windows XP meets these goals by prefetching files necessary to load the operating system.[62]

21.4 System Architecture

Windows XP is modeled on a microkernel architecture—often called a modified microkernel architecture. On the one hand, Windows XP is a modular operating system with a compact kernel that provides base services for other operating system components. However, unlike a pure microkernel operating system, these components (e.g., the file system and memory manager) execute in kernel mode rather than user mode. Windows XP is also a layered operating system, composed of discrete layers with lower layers providing functionality for higher layers.[63] However, unlike a pure layered operating system, there are times when non-adjacent layers communicate. For a description of microkernel and layered operating system designs, see Section 1.13, Operating System Architectures.

Figure 21.1 illustrates Windows XP's system architecture. The **Hardware Abstraction Layer (HAL)** interacts directly with the hardware, abstracting hardware specifics to the rest of the system. The HAL handles device components on the mainboard such as the cache and system timers. The HAL abstracts hardware specifics that might differ between systems of the same architecture, such as the exact layout of the mainboard. In many cases, kernel-mode components do not access the hardware directly, but instead call functions exposed by the HAL. Therefore, kernel-mode components can be constructed with little regard for architecture-specific details such as cache sizes and the number of processors included. The HAL interacts with device drivers to support access to peripheral devices.[64]

The HAL also interacts with the microkernel. The microkernel provides basic system mechanisms, such as thread scheduling, to support other kernel-mode components. *[Note:* The Windows XP microkernel should not be confused with the microkernel design philosophy discussed in Section 1.13, Operating System Architectures.] The microkernel also handles thread synchronization, dispatches interrupts and handles exceptions.[65] Additionally, the microkernel abstracts architecture-specific features—features that differ between two different architectures, such as the number of interrupts. The microkernel and the HAL combine to make Windows XP portable, allowing it to run in many different hardware environments.[66]

The microkernel forms only a small part of kernel space. Kernel space describes an execution environment in which components can access all system memory and services. Components that reside in kernel space execute in kernel mode. The microkernel provides the base services for the other components residing in kernel space.[67]

Layered on top of the microkernel are the kernel-mode components responsible for administering the operating system subsystems (e.g., the I/O manager and virtual memory manager). Collectively, these components are known as the **executive.** Figure 21.1 displays the key executive components. The rest of this chapter is devoted to describing these subsystems and the executive components which man-

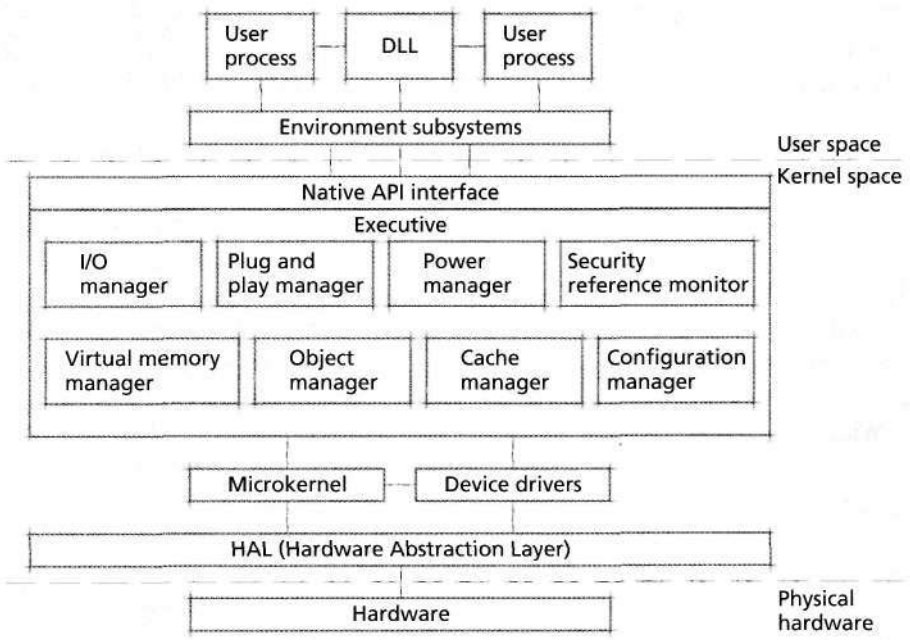


Figure 21.1 | Windows XP system architecture.

age the subsystems. The executive exposes services through an application programming interface (API) to user processes.[68] This API is called the **native API.**

However, most user processes do not call native API functions directly; rather, they call API functions exposed by user-mode system components called **environment subsystems.** Environment subsystems are user-mode processes interposed between the executive and the rest of user space that export an API for a specific computing environment. For example, the **Win32 environment subsystem** provides a typical 32-bit Windows environment. Win32 processes call functions defined in the Windows API; the Win32 subsystem translates these function calls to system calls in the native API (although, in some cases, the Win32 subsystem process itself can fulfill an API request without calling the native API).[69] Although Windows XP allows user-mode processes to call native API functions, Microsoft urges developers to use an environment subsystem's interface.[70]

By default, 32-bit Windows XP includes only the Win32 subsystem, but users can install a POSIX subsystem. Microsoft replaced the POSIX subsystem in older versions of Windows with a more robust UNIX environment, called Service For UNIX (SFU) 3.0. Running SFU 3.0, users can port code written for a UNIX environment to Windows XP. To run 16-bit DOS applications, Windows XP provides a Virtual DOS Machine (VDM), which is a Win32 process that provides a DOS environment for DOS applications.[71]

On Windows XP 64-Bit Edition, the default subsystem supports 64-bit applications. However, Windows XP 64-Bit Edition provides a subsystem called the Windows on Windows 64 (WOW64) subsystem, which allows users to execute 32-bit Windows applications. Windows XP 64-Bit Edition does not support 16-bit applications.[72]

The top layer of the Windows XP architecture consists of user-mode processes. These are typical applications (e.g., word processors, computer games and Web browsers) as well as **dynamic link libraries (DLLs).** DLLs are modules that provide data and functions to processes. As shown in Fig. 21.1, the environment subsystems' APIs are simply DLLs that processes dynamically link when calling a function in a subsystem's API. Because DLLs are dynamically linked, applications can be more modular. If the implementation of a DLL function changes, only the DLL must be recompiled and not the applications that use it.[73]

Windows XP also includes special user-mode processes called system services (or Win32 services), which are similar to Linux daemons (see Section 20.3, Linux Overview). These processes usually execute in the background whether or not a user is logged onto the computer and typically execute the server side of client/server applications.[74] Examples of system services are the Task Scheduler (which allows users to schedule tasks to be executed at specific times), IPSec (which manages Internet security usually during data transfer operations) and Computer Browser (which maintains a list of computers connected on the local network).[75] *[Note:* Sometimes, authors also use the term "system services" to refer to native API functions.]

21.5 System Management Mechanisms

Before we investigate each component in the Windows XP operating system, it is important to understand the environment in which these components execute. This section describes how components and user processes store and access configuration data in the system. We also consider how Windows XP implements objects, how these objects are managed and how user- and kernel-mode threads can manipulate these objects. Finally, we describe how Windows XP prioritizes interrupts and how the system employs software interrupts to dispatch processing.

21.5.1 Registry

The **registry** is a database, accessible to all processes and kernel-mode components that stores configuration information[76] specific to users, applications and hardware. For example, user information might include desktop settings and printer settings. Application configuration includes such settings as recently used menus, user preferences and application defaults. Hardware information includes which devices are currently connected to the computer, which drivers serve each device and which resources (e.g., hardware ports) are assigned to each device. The registry also stores system information such as file associations (e.g., .doc files are associated with

Microsoft Word). Applications, device drivers and the Windows XP system use the registry to store and access data in a centralized manner. For example, when a user installs a hardware device, device drivers place configuration information into the registry. When Windows XP allocates resources to various devices, it accesses the registry to determine what devices are installed in the system and distributes resources accordingly.[77]

Windows XP organizes the registry as a tree, and each node in the tree represents a registry key. A key stores subkeys and values; values consist of a value name and value data.[78] The system provides several predefined keys. For example, HKEY_LOCAL_MACHINE is a predefined key which forms the root of all configuration data for the local computer. Values in the HKEY_LOCAL_MACHINE key or its subkeys store such information as the hardware devices connected to the computer, system memory and network information (e.g., server names). Predefined keys act as roots for a single type of data, such as application-specific data or user preferences.[79]

Threads can access subkeys by navigating the tree structure. Beginning with any key that is open, a thread can enumerate that key's subkeys and open any of them. The system's predefined keys remain open at all times. Once a key is open, values and subkeys can be read, modified, added or deleted (assuming the thread has access rights for the key to accomplish all these actions).[80, 81]

Windows XP stores the registry in a collection of hives, which are portions of the registry tree. Most hives are stored as files and flushed to disk periodically to protect the data against crashes, whereas other hives remain solely in memory (e.g., HKEY_LOCAL_MACHINE\HARDWARE, which stores hardware information that the system collects at boot time and updates as it runs).[82] The **configuration manager** is the executive component responsible for managing the registry. Device drivers, system components and user applications communicate with the configuration manager to modify or obtain data from the registry. The configuration manager is also responsible for managing registry storage in system memory and in the hives.[83]

21.5.2 Object Manager

Windows XP represents physical resources (e.g., peripheral devices) and logical resources (e.g., processes and threads) with objects. Note that these objects do not refer to objects in any object-oriented programming language, but rather constructs that Windows XP uses to represent resources. Windows XP represents objects with a data structure in memory; this data structure stores the object's attributes and procedures.[84] Each object in Windows XP is defined by an object type; the object type is also stored as a data structure. Object types are created by executive components; for example, the I/O manager creates the file object type. The object type specifies the types of attributes an instance of this type possesses and the object's standard procedures. Examples of standard procedures include the open procedure, `close` procedure and `delete` procedure.[85] Object types in Windows XP include files, devices, processes, threads, pipes, semaphores and many others. Each Windows XP object is an instance of an object type.[86]

User-mode processes and kernel components can access objects through **object handles.** An object handle is a data structure that allows processes to manipulate the object. Using an object handle, a thread can call one of the object's procedures or manipulate the object's attributes. A kernel-mode component or user-mode process with sufficient access rights can obtain a handle to an object by specifying the object's name in a call to the object's open method or by receiving a handle from another process.[87] Once a process obtains a handle to an object, this process can duplicate the handle; this can be used by a process that wishes to pass a handle to another process and also retain a handle to the object.[88] Processes can use a handle in similar way as pointers, but a handle gives the system extra control over the actions of a user process (e.g., by restricting what the process can do with the handle, such as not allowing the process to duplicate it).

Kernel-mode components can use handles when interacting with user-mode processes, but otherwise can access objects directly through pointers. In addition, kernel-mode components can create **kernel handles,** which are handles accessible from any process's address space, but only from kernel mode. In this way, a device driver can create a kernel handle to an object and be guaranteed access to that object, regardless of the context in which the driver executes.[89] Objects can be named or unnamed, although only kernel-mode components can open a handle to an unnamed object. A user-mode thread can obtain a handle to an unnamed object only by receiving it from a kernel-mode component.[90]

The **object manager** is the executive component that manages objects for a Windows XP system. The object manager is responsible for creating and deleting objects and maintaining information about each object type, such as how many objects of a certain type exist.[91] The object manager organizes all named objects into a hierarchal directory structure. For example, all device objects are stored in the \Device directory or one of its subdirectories. This directory structure composes the **object manager namespace** (i.e., group of object names in which each name is unique in the group).[92]

To create an object, a process passes a request to the object manager; the object manager initializes the object and returns a handle (or pointer if a kernel-mode component called the appropriate function) to it. If the object is named, the object manager enters it into the object manager namespace.[93] For each object, the object manager maintains a count of how many existing handles refer to it and a count of how many existing references (both handles and pointers) refer to it. When the number of handles reaches zero, the object manager removes the object from the object manager namespace. When the number of references to the object reaches zero, the object manager deletes the object.[94]

21.5.3 Interrupt Reqiest levels (IRQLs)

Interrupt definition and handling are crucial to Windows XP. Because some interrupts, such as hardware failure interrupts, are more important than others, Windows XP defines **interrupt request levels (IRQLs).** IRQLs are a measure of

interrupt priority; a processor always executes at some IRQL, and different processors in a multiprocessor system can be executing at different IRQLs. An interrupt that executes at an IRQL higher than the current one can interrupt the current execution and obtain the processor. The system masks (i.e., delays processing) interrupts that execute at an IRQL equal to or lower than the current one.[95]

Windows XP defines several IRQLs (Fig. 21.2). User- and kernel-mode threads normally execute at the **passive IRQL,** which is the lowest-priority IRQL. Unless user-mode threads are executing asynchronous procedure calls (APCs), which we describe in Section 21.5.4, Asynchronous Procedure Calls (APCs), these threads always execute at the passive IRQL. In some instances, kernel-mode threads execute at a higher IRQL to mask certain interrupts, but typically, the system executes threads at the passive level so that incoming interrupts can be serviced promptly. Executing at the passive IRQL implies that no interrupt is currently being processed and no interrupts are masked. APCs, which are procedure calls that threads or the system can queue for execution by a specific thread, execute at the **APC IRQL.** Above the APC level is the **DPC/dispatch IRQL** at which deferred procedure calls (i.e., software interrupts that can be executed in any thread's context) and various other important kernel functions execute, including thread scheduling.[96] We describe DPCs in Section 21.5.5, Deferred Procedure Calls (DPCs). All software interrupts execute at the APC level or DPC/dispatch level.[97]

Hardware interrupts execute at the remaining higher IRQLs. Windows XP provides multiple **device IRQLs (DIRQLs)** at which interrupts from devices execute.[98] The number of DIRQLs varies, depending on how many interrupts a given architecture supports, but the assignment of these levels to devices is arbitrary; it is dependent on the specific interrupt request line at which a device interrupts. Therefore, Windows XP maps device interrupts to DIRQLs without priority; still, if a processor is executing at a certain DIRQL, interrupts that occur at DIRQLs less than or equal to the current one are masked.[99]

The system reserves the highest five IRQLs for critical system interrupts. The first level above DIRQL is the **profile IRQL,** which is used when kernel profiling is enabled (see Section 14.5.1, Tracing and Profiling). The system clock, which issues interrupts at periodic intervals, interrupts at the **clock IRQL.** Interprocessor requests (i.e., interrupts sent by one processor to another such as when shutting down the system) execute at the **request IRQL.** The **power IRQL** is reserved for power failure notification interrupts, although this level has never been used in any NT-based operating system. Interrupts issued at the **high IRQL** (which is the highest IRQL) include hardware failure interrupts and bus errors.[100]

For each processor, the HAL masks all interrupts with IRQLs less than or equal to the IRQL at which that processor is currently executing. This allows the system to prioritize interrupts and execute critical interrupts as quickly as possible. For example, executing at the DPC/dispatch level masks all other DPC/dispatch level interrupts and APC level interrupts. Interrupts that execute at an IRQL that is currently masked are not discarded (i.e., removed from the system). When the

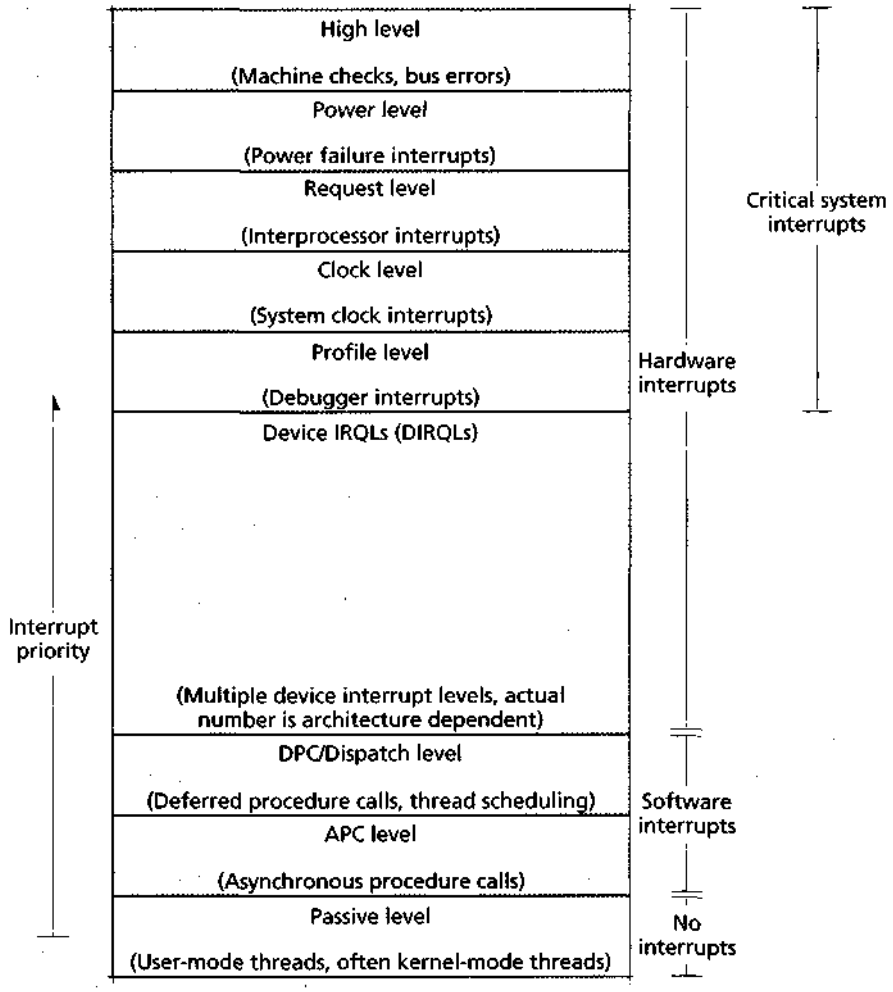


Figure 21.2 | Interrupt request levels (IRQLs)

IRQL drops low enough, the processor processes the waiting interrupts. Kernel-mode threads can raise the IRQL, and a thread that does so is responsible for subsequently lowering the IRQL to its previous level; user-mode threads cannot change the IRQL. Windows XP attempts to do as much processing as possible at the passive level, at which no interrupts are masked, so that all incoming interrupts can be serviced immediately.[101]

21.5.4 Asynchronous Procedure Calls (APCs)

Asynchronous procedure calls (APCs) are procedure calls that threads or the system can queue for execution by a specific thread; the system uses APCs to complete vari-

ous functions that must be done in a specific thread's context. For example, APCs facilitate asynchronous I/O. After initiating an I/O request, a thread can perform other work. When the I/O request completes, the system sends an APC to that thread, which the thread executes to complete the I/O processing. Windows XP uses APCs to accomplish a variety of system operations. For example, APCs are used by the SFU to send signals to POSIX threads and by the executive to notify a thread of an I/O completion or direct a thread to perform an operation.[102]

Each thread maintains two APC queues: one for **kernel-mode APCs** and one for **user-mode APCs.**[103] Kernel-mode APCs are software interrupts that are generated by kernel-mode components, and a thread must process all pending kernel-mode APCs as soon as it obtains a processor. A user-mode thread can create a user-mode APC and request to queue this APC to another thread; the system queues the APC to the target thread if the queuing thread has sufficient access rights. A thread executes user-mode APCs when the thread enters an **alertable *wait* state.**[104] **In** an alertable *wait* state, a thread awakens either when it has a pending APC or when the object or objects on which the thread is waiting become available. A thread can specify that it enters an alertable *wait* state when it waits on an object, or a thread can simply enter an alertable *wait* state without waiting on any object.[105]

Windows XP also differentiates between normal kernel APCs and special kernel APCs. A thread executes kernel APCs in FIFO order, except that the thread executes all special kernel APCs before all normal kernel APCs.[106] When the system queues a kernel APC, the target thread does not execute the APC until the thread gains the processor. However, once the thread begins executing at the passive level, it begins executing kernel APCs until it drains (i.e., empties) its queue.

Unlike kernel APCs, threads can choose when to execute user-mode APCs. A thread drains its user-mode APC queue when the thread enters an alertable *wait* state. If the thread never enters such a state, its pending user-mode APCs are discarded when it terminates.[107]

A thread executing an APC is not restricted, except that it cannot receive new APCs. It can be preempted by another higher-priority thread. Because scheduling is not masked at the APC level (this occurs at the DPC/dispatch level), a thread executing an APC can block or access pageable data (which might cause a page fault and hence cause Windows XP to execute scheduling code at the DPC/dispatch IRQL). APCs can also cause the currently executing thread to be preempted. If a thread queues an APC to a thread of higher priority that is in an alertable *wait* state, the waiting thread awakens, preempts the queuing thread and executes the APC.[108]

21.5.5 Defered Procedure Calls (DPCs)

Most of Windows XP's hardware interrupt processing occurs in **deferred procedure calls (DPCs).** DPCs are software interrupts that execute at the DPC/dispatch IRQL[109] and which run in the context of the currently executing thread. In other words, the routine that creates the DPC does not know the context in which the **DPC** will execute. Therefore, DPCs are meant for system processing, such as interrupt pro-

cessing, that does not need a specific thread context in which to execute.[110] These procedure calls are called "deferred" because Windows XP processes most hardware interrupts by calling the associated interrupt service routine, which is responsible for quickly acknowledging the interrupt and queuing a DPC for later execution. This strategy increases the system's responsiveness to incoming device interrupts.

Windows XP attempts to minimize the time it spends at DIRQLs to maintain high responsiveness to all device interrupts. Recall that the system maps device interrupts arbitrarily to DIRQLs. Still, device interrupts mapped to a high DIRQL can interrupt the processing of a device interrupt mapped to a lower DIRQL. By using DPCs, the system returns to an IRQL below all DIRQLs and therefore can minimize this difference in the level of interrupt servicing.[111]

Device interrupt handlers can schedule DPCs, and Windows XP executes some kernel functions, such as thread scheduling, at the DPC/dispatch IRQL.[112] When an event that prompts scheduling occurs—such as when a thread enters the *ready* state, when a thread enters the *wait* state, when a thread's quantum expires or when a thread's priority changes—the system executes scheduling code at the DPC/dispatch IRQL.[113]

As we noted above, DPCs are restricted because the routine that creates a DPC does not specify the context in which the DPC executes (this context is simply the currently executing thread). DPCs are further restricted because they mask thread scheduling. Therefore, a DPC cannot cause the thread within whose context it is executing to block, because this would cause the system to attempt to execute scheduling code. In this case no thread would execute because the executing thread blocks, but the scheduler does not schedule a new thread because scheduling is masked (in reality, Windows XP halts the system and displays an error message).[114] As a result, DPCs cannot access pageable data because if a page fault occurs, the executing thread blocks to wait for the completion of the data access. DPCs, however, can be preempted by hardware interrupts.[115, 116]

When a device driver generates a DPC, the system places the DPC in a queue associated with a specific processor. The driver routine can request a processor to which the DPC should be queued (e.g., to ensure that the DPC routine is executed on the processor with relevant cached data); if no processor is specified, the processor queues the DPC to itself.[117] The routine can also specify the DPCs priority. Low- and medium-priority DPCs are placed at the end of the processor's DPC queue; high-priority DPCs enter at the front of the queue. In most cases, when the IRQL drops to the DPC/dispatch level, the processor drains the DPC queue. However, when a low-priority DPC has been waiting for only a short time and the DPC queue is short, the processor does not immediately execute the DPC. The next time the processor drains the DPC queue, it processes this DPC. This policy allows the system to return to the passive level faster to unmask all interrupts and resume normal thread processing.[118]

21.5.6 System Threads

When a kernel-mode component, such as a device driver or executive component, performs some processing on behalf of a user thread, the component's functions often execute in the context of the requesting thread. However, in some cases, kernel-mode components must accomplish a task that is not in response to a particular thread request. For example, the cache manager must periodically flush dirty cache pages to disk. Also, a device driver servicing an interrupt might not be able to accomplish all necessary processing at an elevated IRQL (i.e., the device's DIRQL when it interrupts or the DPC/dispatch level when executing a DPC). For example, the driver might need to accomplish some work at the passive IRQL, because it must access pageable data.[119]

Developers of kernel-mode components have two options to perform this processing. A component can create a kernel thread, which is a thread that executes in kernel mode and typically belongs to the System process—a kernel-mode process to which most kernel-mode threads belong. Aside from executing in kernel mode, kernel threads behave and are scheduled the same as user threads.[120] Kernel threads normally execute at the passive IRQL, but can raise and lower the IRQL.[121]

A second option is to use a **system worker thread**—a thread, which is created at system initialization time or dynamically in response to a high volume of requests, that sleeps until a work item is received. A work item consists of a function to execute and a parameter describing the context in which to execute that function. System worker threads also belong to the System process. Windows XP includes three types of worker threads: delayed, critical and hypercritical. The distinction arises from the scheduling priority given to each type. Delayed threads have the lowest priority of the three types; critical threads have a relatively high priority; and the hypercritical thread (there is only one), which is reserved for system use (e.g., device drivers cannot queue work items to this thread), has the highest priority of the three types.[122, 123]

21.6 Process and Thread Management

In Windows XP, a process consists of program code, an execution context, resources (e.g., object handles) and one or more associated threads. The execution context includes such items as the process's virtual address space and various attributes (e.g., security attributes and working set size limitations). Threads are the actual unit of execution; threads execute a piece of a process's code in the process's context, using the process's resources. A thread also contains its own execution context which includes its runtime stack, the state of the machine registers and several attributes (e.g., scheduling priority).[124] Both processes and threads are objects, so other processes can obtain handles for processes and threads, and other threads can wait on process and thread events just as with other object types.[125, 126]

21.6.1 Process and Thread Organization

Windows XP stores process and thread context information in several data structures. An **executive process block (EPROCESS block)** is the main data structure that describes a process. The EPROCESS block stores information that executive components use when manipulating a process object; this information includes the process's ID, a pointer to the process's handle table, a pointer to the process's access token and working set information (e.g., the process's minimum and maximum working set size, page fault history and the current working set). The system stores EPROCESS blocks in a linked list.[127, 128]

Each EPROCESS block also contains a **kernel process block (KPROCESS block).** A KPROCESS block stores process information used by the microkernel. Recall that the microkernel manages thread scheduling and thread synchronization. Therefore, the KPROCESS block stores such information as the process's base priority class, the default quantum for each of its threads and its spin lock.[129, 130]

The EPROCESS and KPROCESS blocks exist in kernel space and store information for kernel-mode components to access while manipulating a process. The system also stores process information in a **process environment block (PEB),** which is stored in the process's address space. A process's EPROCESS block points to the process's PEB. The PEB stores information useful for user processes, such as a list of DLLs linked to the process and information about the process's heap.[131, 132]

Windows XP stores thread data in a similar manner. An **executive thread block (ETHREAD block)** is the main data structure describing a thread. It stores information that executive components use when manipulating a thread object. This information includes the ID of the thread's process, its start address, its access token and a list of its pending I/O requests. The ETHREAD block for a thread also points to the EPROCESS block of its process.[133, 134]

Each ETHREAD block stores a **kernel thread block (KTHREAD block).** The microkernel uses information in the KTHREAD block for thread scheduling and synchronization. For example, the KTHREAD block stores information such as the thread's base and current priority (later in this section we see how a thread's priority can change), its current state (e.g., *ready, waiting)* and any synchronization objects for which it is waiting.[135, 136]

The ETHREAD and KTHREAD blocks exist in kernel space and therefore are not accessible to users. A **thread environment block (TEB)** stores information about a thread in the thread's process's address space. Each KTHREAD block points to a TEB. A thread's TEB stores information such as the critical sections owned by the thread, its ID and information about its stack. A TEB also points to the PEB of the thread's process.[137, 138]

All threads belonging to the same process share a virtual address space. Although most of this memory is global, threads can maintain their own data in **thread local storage (TLS).** A thread within a process can allocate a TLS index for the process; the thread stores a pointer to local data in a specified location (called a **TLS slot)** in this index. Each thread using the index receives one TLS slot in the

index in which it can store a data item. A common use for a TLS index is to store data associated with a DLL that a process links. Processes often contain many TLS indexes to accommodate multiple purposes, such as for DLL and environment subsystem data. When threads no longer need a TLS index (e.g., the DLL completes execution), the process can discard the index.[139] A thread also possesses its own runtime stack on which it can also store local data.

Creating and Terminating Processes

A process can create another process using API functions. For Windows processes, the parent (i.e., creating) process and child (i.e., created) process are completely independent. For example, the child process receives a completely new address space.[140] This differs from the `fork` command in Linux, in which the child receives a copy of its parent's address space. The parent process can specify certain attributes that the child process **inherits** (i.e., the child acquires a duplicate from the parent), such as most types of handles, environment variables—i.e., variables that define an aspect of the current settings, such as the operating system version number—and the current directory.[141] When the system initializes a process, the process creates a **primary thread.** The primary thread acts as any other thread, and any thread can create other threads belonging to that thread's process.[142]

A process can terminate execution for a variety of reasons. If all of its threads terminate, a process terminates, and any of a process's threads can explicitly terminate the process at any time. Additionally, when a user logs off, all the processes running in the user's context are terminated. Because parent and child processes are independent of one another, terminating a parent process has no affect on its children and vice versa.[143]

Jobs

Sometimes, it is preferable to group several processes together into a unit, called a job. A **job object** allows developers to define rules and set limits on a number of processes. For example, a single application might consist of a group of processes. A developer might want to restrict the number of processor cycles and amount of memory that the application consumes (e.g., so that an application executing on behalf of a client does not monopolize all of a server's resources). Also, the developer might want to terminate all the processes of a group at once, which is difficult because all processes are normally independent of one another.[144] A process can be a member of at most one job object. The job object specifies such attributes as a base priority class, a security context, a working set minimum and maximum size, a virtual memory size limit and both a per-process and jobwide processor time limit. Just as a thread inherits these attributes from the process to which it belongs, a process inherits these attributes from its associated job (if the process belongs to a job).[145,146] The system can terminate all processes in a job by terminating the job.[147]

Systems executing batch processes, such as data mining, benefit from jobs. Developers can limit the amount of processor time and memory a computationally intensive job consumes to free more resources for interactive processes. Sometimes

it is useful to create a job for a single process, because a job allows the developer to specify tighter restrictions than a process allows. For example, with a job object, a developer can limit the process's total processor time; a developer cannot use the process object to do this.[148]

Fibers

Threads can create **fibers,** which are similar to threads, except that a fiber is scheduled for execution by the thread that creates it, rather than the microkernel. Windows XP includes fibers to permit the porting of code written for other operating systems to Windows XP. Some operating systems, such as many UNIX varieties, schedule processes and require them to schedule their own threads. Developers can use fibers to port code to and from these operating systems. A fiber executes in the context of the thread that creates the fiber.[149]

Creating fibers generates additional units of execution within a single thread. Each fiber must maintain state information, such as the next instruction to execute and the values in a processor's registers. The thread stores this state information for each fiber. The thread itself is also a unit of execution, and thus must convert itself into a fiber to separate its own state information from other fibers executing in its context. In fact, the Windows API forces a thread to convert itself into a fiber before creating or scheduling other fibers. Although the thread becomes a fiber, the thread's context remains, and all fibers associated with that thread execute in that context.[150]

Whenever the kernel schedules a thread that has been converted to a fiber for execution, the converted fiber or another fiber belonging to that thread runs. Once a fiber obtains the processor, it executes until the thread in whose context the fiber executes is preempted, or the fiber switches execution to another fiber (either within the same thread or a fiber created by a separate thread). Just as threads possess their own thread local storage (TLS), fibers possess their own **fiber local storage (FLS),** which functions for fibers exactly as TLS functions for a thread. A fiber can also access its thread's TLS. If a fiber deletes itself (i.e., terminates), its thread terminates.[151]

Fibers permit Windows XP applications to write code executed using the many-to-many mapping; typical Windows XP threads are implemented with a one-to-one mapping (see Section 4.6.2, Kernel-Level Threads). Fibers are user-level units of execution and invisible to the kernel. This makes context switching between fibers of the same thread fast because it is done in user mode. Therefore, a single-threaded process with many fibers can simulate a multithreaded process. However, the Windows XP microkernel only recognizes one thread, and because Windows XP schedules threads, not processes, the single-threaded process receives less execution time (all else being equal).[152, 153]

Thread Pools

Windows XP also provides each process with a **thread pool** that consists of a number of **worker threads,** which execute functions queued by user threads. The worker

threads sleep until a request is queued to the pool.[154] The thread that queues the request must specify the function to execute and must provide context information.[155] When a process is created, it receives an empty thread pool; the system allocates space for worker threads when the process queues its first request.[156]

Thread pools have many purposes. Developers can use them to handle client requests. Instead of incurring the costly overhead of creating and destroying a thread for each request, the developer simply queues the request to the thread pool. Also, several threads that spend most of their time sleeping (e.g., waiting for events to occur) can be replaced by a single worker thread that awakens each time one of these events occurs. Furthermore, applications can use the thread pool to accomplish asynchronous I/O by queuing a worker thread to execute the I/O completion routines. Using thread pools can make an application more efficient and simpler because developers do not have to create and delete as many threads. However, thread pools transfer some control from the programmer to the system, which can introduce inefficiency. For example, the system grows and shrinks the size of a process's thread pool in response to request volume; in some cases, the programmer can better guess how many threads are needed. Thread pools also require memory overhead, because the system grows a process's (initially empty) thread pool as the process's threads queue work items.[157]

21.6.2 Thread Scheduling

Windows XP does not contain a specific "thread scheduler" module—the scheduling code is dispersed throughout the microkernel. The scheduling code collectively is referred to as the **dispatcher.**[158] Windows XP supports preemptive scheduling among multiple threads. The dispatcher schedules each thread without regard to the process to which the thread belongs. This means that, all else being equal, the same process implemented with more threads will get more execution time.[159] The scheduling algorithm used by Windows XP is based on a thread's priority. Before describing the scheduling algorithm in detail, we investigate the lifecycle of a Windows XP thread.

Thread States

In Windows XP, threads can be in any one of eight states (Fig. 21.3). A thread begins in the ***initialized*** **state** during thread creation. Once initialization concludes, the thread enters the ***ready*** **state.** Threads in the ready state are waiting to use a processor. A thread that the dispatcher has decided will execute on a particular processor next enters the ***standby*** **state** as it awaits its turn for that processor. A thread is in the standby state, for example, during the context switch from the previously executing thread to that thread. Once the thread obtains a processor, it enters the ***running*** **state.** A thread transitions out of the *running* state if it terminates execution, exhausts its quantum, is preempted, is suspended or waits on an object. If a thread terminates, it enters the ***terminated*** **state.** The system does not necessarily delete a *terminated* thread; the object manager deletes a thread only when the thread object's

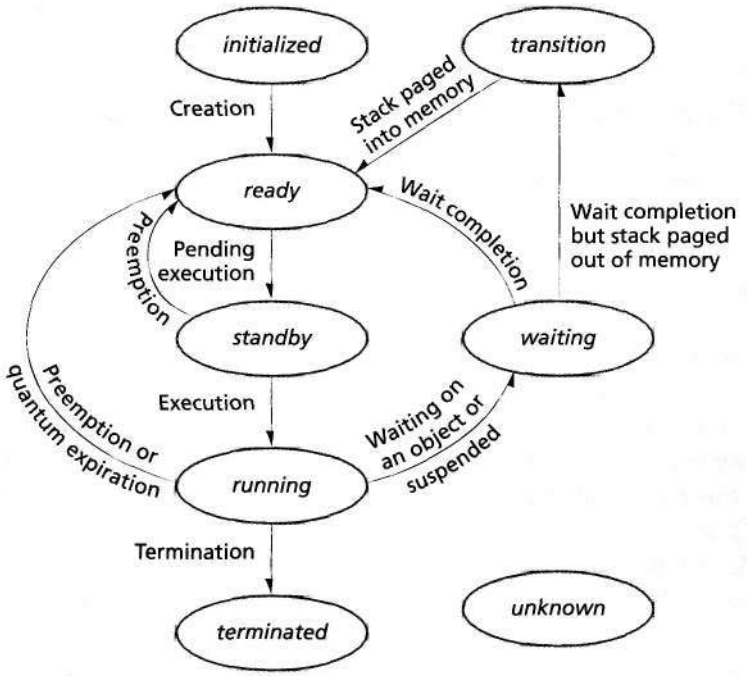


Figure 21.3 | Thread state-transition diagram.

reference count becomes zero. If a *running* thread is preempted or exhausts its quantum, it returns to the *ready* state. If a *running* thread begins to wait on an object handle, it enters the ***waiting* state.** Also, another thread (with sufficient access rights) or the system can suspend a thread, forcing it into the *waiting* state until the thread is resumed. When the thread completes its wait, it either returns to the *ready* state or enters the ***transition* state.** A thread in the *transition* state has had its kernel stack paged out of memory (e.g., because it has not executed in a while and the system needed the memory for other purposes), but the thread is otherwise ready to execute. The thread enters the *ready* state once the system pages the thread's kernel stack back into memory. The system places a thread in the ***unknown* state** when the system is unsure of the thread's state (usually because of an error).[160, 161]

Thread Scheduling Algorithm

The dispatcher schedules threads based on priority—the next subsection describes how thread priorities are determined. When a thread enters the *ready* state, the kernel places it into the ready queue that corresponds to its priority. Windows XP has 32 priority levels, denoted by integers from 0 through 31, where 31 is the highest priority and 0 is the lowest. The dispatcher begins with the highest-priority ready queue and schedules the threads in this queue in a round-robin fashion. A thread

remains in a ready queue as long as the thread is in either the *ready* state or the *running* state. Once the queue empties, the dispatcher proceeds to the next queue; it continues in this manner until either all queues are empty or a thread with a higher priority than the currently executing thread enters its *ready* state. In the latter case, the dispatcher preempts execution of the lower-priority thread and executes the new thread.[162] The dispatcher then executes the first thread in the highest-priority nonempty ready queue and resumes its normal scheduling procedure.[163]

Each thread executes for at most one quantum. In the case of preemption, real-time threads have their quantum reset, whereas all other threads finish their quantum when they reacquire a processor. Giving real-time threads fresh quanta after preemption allows Windows XP to favor real-time threads, which require a high level of responsiveness. The system returns preempted threads to the front of the appropriate ready queue.[164] Note that user-mode threads can preempt kernel-mode threads in many cases. However, kernel-mode threads can prevent this by masking certain interrupts. Events such as a thread entering the *ready* state, a thread exiting the *running* state or a change in a thread's priority trigger the system to execute dispatcher routines—routines that execute at DPC/dispatch level. By raising the IRQL to the DPC/dispatch IRQL, kernel-mode threads can mask scheduling and avoid being preempted. However, user-mode threads can still block the execution of system threads if they have a higher priority than the system threads.[165]

Determining Thread Priority

Windows XP divides its 32 priority levels (0-31) into two categories. Real-time threads (i.e., threads that must maintain a high level of responsiveness to user requests) occupy the upper 16 priority levels (16-31), and dynamic threads occupy the lower 16 priority levels (0-15). Only the zero-page thread has priority level 0. This thread uses spare processor cycles to zero free memory pages so they are ready for use.

Each thread has a **base priority,** which defines the lower limit that its actual priority may occupy. A user-mode thread's base priority is determined by its process's base priority class and the thread's priority level. A process's **base priority class** specifies a narrow range that the base priority of each of a process's threads can have. There are six base priority classes: idle, below normal, normal, above normal, high and real-time. The first five of these priority classes (called the **dynamic priority classes)** encompass priority levels 0 through 15; these are called dynamic priority classes, because threads belonging to processes in these classes can have their priorities dynamically altered by the operating system. The threads belonging to processes in the **real-time priority class** have a priority between 16 and 31; real-time threads have a static priority. Within each priority class, there are several **base priority levels:** idle, lowest, below normal, normal, above normal, highest and time critical. Each combination of base priority class and base priority level maps to a specific base priority (e.g., a thread with a normal base priority level and a normal priority class has a base priority of 7).[166]

A dynamic-priority thread's priority can change. A thread receives a priority boost when the thread exits the *waiting* state, such as after the completion of an I/O or after the thread gains a handle to a resource for which the thread is waiting. Similarly, a window that receives input (such as from the keyboard, mouse or timer) gets a priority boost. The system also can reduce a thread's priority. When a thread executes until its quantum expires, its priority is reduced one unit. However, a thread's dynamic priority can never dip below its base priority or rise into the real-time range.[167]

Finally, to reduce the likelihood of threads being indefinitely postponed, the dispatcher periodically (every few seconds) scans the lists of ready threads and boosts the priority of dynamic threads that have been waiting for a long time. This scheme also helps solve the problem of **priority inversion.** Priority inversion occurs when a high-priority thread is prevented from gaining the processor by a lower-priority thread. For example, a high-priority thread might wait for a resource held by a low-priority thread. A third, medium-priority thread obtains the processor, preventing the low-priority thread from running. In this way, the medium-priority thread is also preventing the high-priority thread from executing—hence, priority inversion. The dispatcher will eventually boost the priority of the low-priority thread so it can execute and, hopefully, finish using the resource.[168]

Scheduling on Multiprocessors

Multiprocessor thread scheduling extends the preceding uniprocessor scheduling algorithm. All editions of Windows XP, except the Home Edition, provide support for multiple processors. However, even Home Edition provides support for symmetric multiprocessing (SMP) using Intel's hyper-threading (HT) technology. Recall from Section 15.2.1 that HT technology allows the operating system to view one physical processor as two virtual processors. Scheduling in an SMP system is similar to that in a uniprocessor system. Generally, when a processor becomes available, the dispatcher tries to schedule the thread at the front of the highest-priority nonempty ready queue. However, the system also attempts to keep threads on the same processors to maximize the amount of relevant data stored in L1 caches (see Section 15.4, Memory Access Architectures).

Processes and threads can specify the processors on which they prefer to execute. A process can specify an **affinity mask,** which is a set of processors on which its threads are allowed to execute. A thread can also specify an affinity mask, which must be a subset of its process's affinity mask.[169] Similarly, a job also can have an affinity mask, which each of its associated processes must set as its own affinity mask. One use of affinity masks is restricting computationally intensive processes to a few processors so these processes do not interfere with more time-critical interactive processes.

In addition to affinity masks, each thread stores its **ideal processor** and **last processor.** Manipulating the value of a thread's ideal processor allows developers to influence whether related threads should execute in parallel (by setting related

threads' ideal processors to different processors) or on the same processor to share cached data (by setting the threads' ideal processors to the same processor). By default, Windows XP attempts to assign threads of the same process different ideal processors. The dispatcher uses the last processor in an effort to schedule a thread on the same processor on which the thread last executed. This strategy increases the likelihood that data cached by the thread during an execution can be accessed during the next execution of that thread.[170, 171]

When a processor becomes available, the dispatcher schedules threads by considering each thread's priority, ideal processor, last processor and how long a thread has been waiting. Consequently, the thread at the front of the highest-priority nonempty ready queue might not be scheduled to the next available processor (even if the processor is in the thread's affinity mask). If this thread has not been waiting long and the available processor is not its ideal or last processor, the dispatcher might choose another thread from that ready queue which meets one or more of these other criteria.[172]

21.6.3 Thread Synchronization

Windows XP provides a rich variety of synchronization objects and techniques, designed for various purposes. For example, some synchronization objects execute at the DPC/dispatch level. These objects provide certain guarantees—a thread holding one of these objects will not be preempted—but restrict functionality (e.g., a thread holding one of these objects cannot access pageable data). Some synchronization objects are designed specifically for kernel-mode threads, whereas others are designed for all threads. In the following subsections, we introduce some synchronization mechanisms provided in Windows XP, how they are used and their benefits and drawbacks.

Dispatcher Objects

Windows XP provides a number of **dispatcher objects** that kernel- and user-mode threads can use for synchronization purposes. Dispatcher objects provide synchronization for resources such as shared data structures or files. These objects include many familiar synchronization constructs such as mutexes, semaphores, events and timers. Kernel threads use kernel dispatcher objects; synchronization objects available to user processes encapsulate these kernel dispatcher objects (i.e., synchronization objects translate many API calls made by user-mode threads into function calls for kernel dispatcher objects). A thread holding a dispatcher object executes at the passive IRQL.[173] A developer can use these objects to synchronize the actions of threads of the same process or different processes.[174]

Dispatcher objects can be in either a ***signaled* state** or ***unsignaled* state** (some dispatcher objects also have states relating to error conditions). The object remains in the *unsignaled* state while the resource for which the dispatcher object provides synchronization is unavailable. Once the resource becomes available, the object enters its *signaled* state. If a thread wishes to access a protected resource, the thread can call a **wait function** to wait for one or more dispatcher objects to enter a *sig-*

naled state. When calling a wait function for a single object, the thread specifies the object on which it is waiting (by passing the object's handle) and a maximum wait time for the object. Multiple-object wait functions can be used when a thread must wait on more than one object. The thread can specify whether it is waiting for all the objects or any one of them. Windows XP supplies numerous variations to these generic wait functions; e.g., a thread can specify that it enters an alertable *wait* state, allowing it to process any APCs queued to it while waiting. After calling a wait function, a thread blocks and enters its *wait* state. When the required object enters its *signaled* state, one or more threads can access the resource (later in this section, we describe how a dispatcher object enters its *signaled* state).[175] In many cases, the kernel maintains FIFO queues for waiting resources, but kernel APCs can disrupt this ordering. Threads process kernel APCs immediately, and when a thread resumes its wait, it is placed at the end of the queue.[176]

Event Objects

Threads often synchronize with an event, such as user input or I/O completion, by using an **event object.** When the event occurs, the object manager sets the event object to the *signaled* state. The event object returns to the *unsignaled* state by one of two methods (specified by the object's creator). In the first method, the object manager sets the event object to the *unsignaled* state when one thread is released from waiting. This method can be used when only one thread should awaken (e.g.. to process the completion of an I/O operation). In the other method, the event object remains in the *signaled* state until a thread specifically sets the event to its *unsignaled* state; this allows multiple threads to awaken. This option can be used, for example, when multiple threads are waiting to read data. When a writing thread completes its operation, the writer can use an event object to signal all the waiting threads to awaken. The next time a thread begins writing, the thread can reset the event object to the *unsignaled* state.[177]

Mutex Objects

Mutex objects provide mutual exclusion for shared resources; they are essentially binary semaphores (see Section 5.6, Semaphores). Only one thread can own a mutex at a time, and only that thread can access the resource associated with the mutex. When a thread finishes using the resource protected by the mutex, it must release the mutex to transition it into its *signaled* state. If a thread terminates before releasing a mutex object, the mutex is considered **abandoned.** In this case the mutex enters an *abandoned* state. A waiting thread can acquire this abandoned mutex. However, the system cannot guarantee that the resource protected by the mutex is in a consistent state. To be safe, a thread should assume an error has occurred if it acquires an abandoned mutex.[178]

Semaphore Objects

Semaphore objects extend the functionality of mutex objects; they allow the creating thread to specify the maximum number of threads that can access a shared

resource.[179] For example, a process might have a small number of preallocated buffers; the process can use a semaphore to synchronize access to these buffers.[180] Semaphore objects in Windows XP are counting semaphores (see Section 5.6.3, Counting Semaphores). A semaphore object maintains a count, which is initialized to the maximum number of threads that can simultaneously access the pool of resources. The count is decremented every time a thread acquires access to the pool of resources protected by the semaphore and incremented when a thread releases the semaphore. The semaphore remains in its *signaled* state while its count is greater than zero; it enters the *unsignaled* state when the count drops to zero.

A single thread can decrement the count by more than one by specifying the semaphore object in multiple wait functions. For example, a thread might use several of the process's buffers in separate asynchronous I/O operations. Each time the thread issues a buffered I/O request, the thread reacquires the semaphore (i.e., by specifying the semaphore object in a wait function), which decrements the semaphore's count. The thread must release the semaphore, which increments the semaphore's count, each time a request completes.[181]

Waitable Timer Objects

Threads might need to perform operations at regular intervals (e.g., autosave a user's document) or at specific times (e.g., display a calendar item). For this type of synchronization, Windows XP provides **waitable timer objects.** These objects become *signaled* after a specified amount of time elapses. **Manual-reset timer objects** remain *signaled* until a thread specifically resets the timer. **Auto-reset timer objects** remain *signaled* only until one thread finishes waiting on the object. Waitable timer objects can be **single use,** after which they become deactivated, or they can be **periodic.** In the latter case, the timer object reactivates after a specified interval and becomes *signaled* once the specified time expires. Figure 21.4 lists Windows XP's dispatcher objects.[182]

Several other Windows XP objects, such as console input, jobs and processes, can function as dispatcher objects as well. For example, jobs, processes and threads are set to the *signaled* state when they terminate, and a console input object is set to the *signaled* state when the console's input buffer contains unread data. In these cases, threads use wait functions and treat these objects just like other dispatcher objects.[183]

Dispatcher Object	*Transitions from Unsignaled to Signaled State When*
Event	Associated event occurs.
Mutex	Owner of the mutex releases the mutex.
Semaphore	Semaphore's count rises above zero.
Waitable timer	Specified amount of time elapses.

Figure 21.4 | Dispatcher objects in Windows XP.

KernelModeLocks

Windows XP provides several mechanisms for synchronizing access to kernel data structures. If a thread is interrupted while accessing a shared data structure, it might leave the data structure in an inconsistent state. Unsynchronized access to a data structure can result in erroneous results. If the thread belongs to a user process, an application might malfunction; if the thread belongs to a kernel process, the system might crash. In a uniprocessor system, one solution to the synchronization problem is to raise the IRQL level above the one at which any component that might preempt the thread executes and access the data structure. The thread lowers the IRQL level when it completes executing the critical code.[184]

Raising and lowering the IRQL level is inadequate in a multiprocessor system—two threads, executing concurrently on separate processors, can attempt to access the same data structure simultaneously. Windows XP provides spin locks to address this problem. Threads holding spin locks execute at the DPC/dispatch level or DIRQL, ensuring that the holder of the spin lock is not preempted by another thread. Threads should execute the fewest instructions possible while holding a spin lock to reduce wasteful spinning by other threads.

When a thread attempts to access a resource protected by a spin lock, it requests ownership of the associated spin lock. If the resource is available, the thread acquires the lock, accesses the resource, then releases the spin lock. If the resource is not available, the thread keeps trying to acquire the spin lock until successful. Because threads holding spin locks execute at the DPC/dispatch level or DIRQL, developers should restrict the actions these threads perform. Threads holding spin locks should never access pageable memory (because a page fault might occur), cause a hardware or software exception, attempt any action that could cause deadlock (e.g., attempt to reacquire that spin lock or attempt to acquire another spin lock unless the thread knows that this will not cause deadlock) or call any function that performs any of these actions.[185]

Windows XP provides generic spin locks and **queued spin locks.** A queued spin lock enforces FIFO ordering of requests and is more efficient than a normal spin lock,[186] because it reduces the processor-to-memory bus traffic associated with the spin lock. With a normal spin lock, a thread continually attempts to acquire the lock by executing a test-and-set instruction (see Section 5.5.2, Test-and-Set Instruction) on the memory line associated with the spin lock. This creates bus traffic on the processor-to-memory bus. With a queued spin lock, a thread that releases the lock notifies the next thread waiting for it. Threads waiting for queued spin locks do not create unnecessary bus traffic by repeating test-and-set. Queued spin locks also increase fairness by guaranteeing FIFO ordering of requests, because the releasing thread uses a queue associated with the spin lock to notify the appropriate waiting thread.[187] Windows XP supports both spin locks (for legacy compatibility) and queued spin locks, but Microsoft recommends the use of the latter.[188]

Alternatively, kernel-mode threads can use **fast mutexes,** a more efficient variant of mutexes that operate at the APC level. Although they require less overhead

than mutexes, fast mutexes somewhat restrict the actions of the thread owning the lock. For example, threads cannot specify a maximum wait time to wait for a fast mutex—they can either wait indefinitely or not wait at all for an unavailable fast mutex. Also, if a thread attempts to acquire a fast mutex that it already owns, it creates a deadlock (dispatcher mutexes allow a thread to reacquire a mutex the thread already owns as long as it releases the mutex for each acquisition of the mutex).[189] Fast mutexes operate at the APC IRQL, which masks APCs that might require the thread to attempt to acquire a fast mutex it already owns.[190] However, some threads might need to receive APCs. For example, a thread might need to process the result of an asynchronous I/O operation; the system often uses APCs to notify threads that an asynchronous I/O request has been processed. For this purpose, Windows XP provides a variant of fast mutexes that execute at the passive IRQL.[191]

Another synchronization resource available only to kernel-mode threads is the **executive resource lock.** Executive resource locks have two modes: shared and exclusive. Any number of threads can simultaneously hold an executive resource lock in shared mode, or one thread can hold an executive resource lock in exclusive mode. This lock is useful for solving the readers-and-writers problem (see Section 6.2.4, Monitor Example: Readers and Writers). A reading thread can gain shared access to a resource if no other thread currently holds or is waiting for exclusive (i.e., write) access to the resource. A writing thread can gain exclusive access to a resource as long as no other thread holds either shared (i.e., read) or exclusive (i.e., write) access to it.[192]

Other Available Synchronization Tools

Aside from dispatcher objects and kernel-mode locks, Windows XP provides to threads several other synchronization tools. Kernel-mode locks are not applicable in all situations—they cannot be used by user-mode threads, and many of them operate at an elevated IRQL, restricting what threads can do while holding them. Dispatcher objects facilitate general synchronization but are not optimized for specific cases. For example, dispatcher objects can be used by threads of different processes but are not optimized for use among threads of the same process.

Critical section objects provide services similar to mutex objects but can be employed only by threads within a single process. Moreover, critical section objects do not allow threads to specify a maximum wait time (it is possible to wait indefinitely for a critical section object), and there is no way to determine if a critical section object is abandoned. However, critical section objects are more efficient than mutex objects because critical section objects do not transition to kernel mode if there is no contention. This optimization requires that critical section objects be employed only to synchronize threads of the same process. Also, critical section objects are implemented with a processor-specific test-and-set instruction, further increasing efficiency.[193,194]

A **timer-queue timer** is an alternate method for using waitable timer objects that allows threads to wait on timer events while executing other pieces of code. When a thread wishes to wait on a timer, it can place a timer in a timer queue and

specify a function to perform when the timer becomes *signaled.* At that time, a worker thread from a thread pool performs the specified function. In cases where the thread waits on both a timer and another object, the thread should use a waitable timer object with a wait function.[195]

For variables shared among multiple threads, Windows XP provides **interlocked variable access** through certain functions. These functions provide atomic read and write access to variables, but they do not ensure the order in which accesses are made. For example, the InterlockedIncrement function combines incrementing a variable and returning the result into a single atomic operation. This can be useful, for example, when providing unique ids. Each time a thread needs to obtain a unique id, it can call InterlockedIncrement. This avoids a situation in which one thread increments the value of the id variable but is interrupted before checking the result. The interrupting thread might then access the id variable, causing two items to have the same id. Because InterlockedIncrement both increments the variable and returns the result in an atomic operation, this situation will not occur.[196] Windows XP also provides **interlocked singly linked lists (SLists).** which provide atomic insertion and deletion operations.[197]

This section provided a broad survey of Windows XP's synchronization tools. However, Windows XP provides other tools; readers interested in more information should visit the MSDN Library at msdn.microsoft.com/library. In addition to these synchronization tools, threads can synchronize by communicating through various IPC mechanisms (described in Section 21.10, Interprocess Communication) or by queuing APCs to other threads.

21.7 Memory Management

The Windows XP **virtual memory manager (VMM)** creates the illusion that each process has a 4GB contiguous memory space. Because the system allocates more virtual memory to processes than can fit into main memory, the VMM stores some of the data on disk in files called **pagefiles.** Windows XP divides virtual memory into fixed-size pages which it stores either in page frames in main memory or files on disk. The VMM uses a two-level hierarchical addressing system.

Windows XP has two strategies for optimizing main memory usage and reducing disk I/O. The VMM uses copy-on-write pages (see Section 10.4.6, Sharing in a Paging System) and employs lazy **allocation,** which is a policy of postponing allocating pages and page table entries in main memory until absolutely necessary. However, when the VMM is forced to perform disk I/O, it prefetches pages from disk and places pages into main memory before they are needed (see Section **11.4.** Anticipatory Paging). Heuristics ensure that the gain in the rate of disk I/O outweighs the cost of filling main memory with potentially unused pages. When main memory fills, Windows XP performs a modified version of the LRU page-replacement algorithm. The following sections describe the internals of the Windows XP memory management system.

21.7.1 Memory Organization

Windows XP systems provide either a 32-bit or a 64-bit address space, depending on the processor and the edition of Windows XP. We limit our discussion to memory operations using the Intel IA-32 architecture (e.g., Intel Pentium and AMD Athlon systems), because the vast majority of today's Windows systems are built for that platform. See the MSDN Library for more information on Windows XP 64-Bit Edition.[198]

Windows XP allocates a unique 4GB virtual address space to each process. By default, a process can access only the first 2GB of its virtual address space. The system reserves the other two gigabytes for kernel-mode components—this space is called system space.[199]

Physical memory is divided into fixed-size page frames, which are 4KB on a 32-bit system (except in the case when the system uses large pages as described later in this section). The VMM maintains a two-level memory map in the system portion of each process's virtual address space that stores the location of pages in main memory and on disk.[200] The VMM assigns each process one **page directory table.** When a processor switches contexts, it loads the location of the new process's page directory table into the **page directory register.** The page directory table is composed of page directory entries (PDEs); each PDE points to a **page table.** Page tables contain page table entries (PTEs); each PTE points to a page frame in main memory or a location on disk. The VMM uses the virtual address in conjunction with the memory map to translate virtual addresses into physical addresses. A virtual address is composed of three portions; the offset in a page directory table, the offset in a page table and the offset on a page in physical memory[201]

Windows XP translates a virtual address in three stages, as shown in Fig. 21.5. In the first stage of address translation, the system calculates the sum $a + d$, which is the value in the page directory register plus the first portion of the virtual address, to determine the location of the page directory entry (PDE) in the page directory table. In the second stage, the system calculates the sum $b + t$, which is the value in the PDE plus the second portion of the virtual address, to find the page table entry (PTE) in the page table. This entry contains c, the page frame number corresponding to the virtual page's location in main memory. Finally, in the last stage, the system concatenates c and the page offset, o, to form the physical address (see Section 10.4.4, Multilevel Page Tables). The system performs all of these operations quickly; the delay occurs when the system must read the PDE and PTE from main memory.[202] Address translation is often accelerated by the Translation Look-aside Buffer (TLB) as discussed in Section 10.4.3, Paging Address Translation with Direct/Associative Mapping.

The PTE is structured differently depending on whether it points to a page in memory or a page in a pagefile. Five of the PTE's 32 bits are for protection—they indicate whether a process may read the page, write to the page and execute code on the page. These protection bits also tell the system whether the page is a copy-on-write page and whether the system should raise an exception when a process

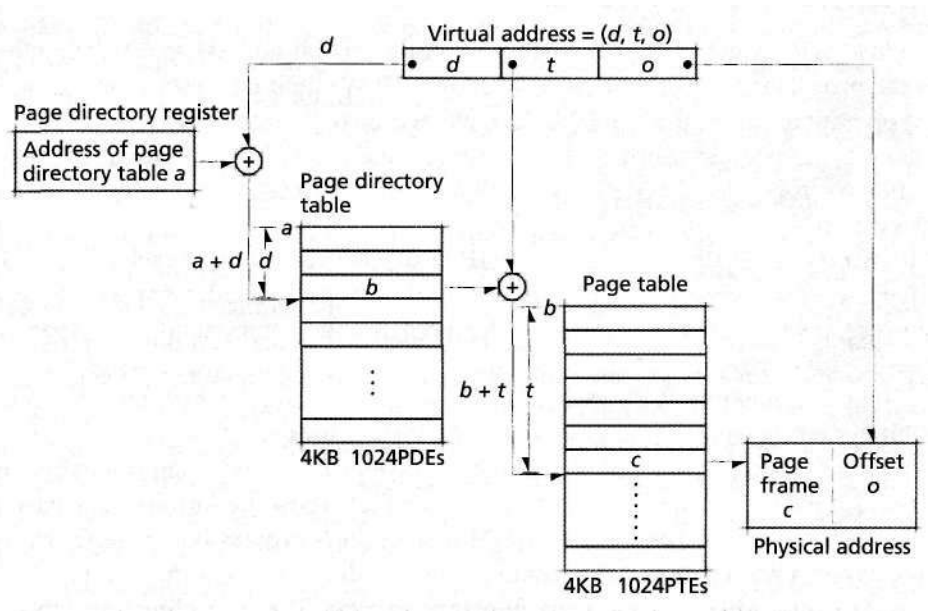


Figure 21.5 | Virtual address translation.

accesses the page.[203] Twenty bits index a frame in memory or offset in a pagefile on disk; this is enough to address 1,048,576 virtual pages. If the page is stored on disk, a designated four bits indicate in which of 16 possible pagefiles the page is located. If the page is in memory, a different set of three bits indicate the state of the page. Figure 21.6 lists these three bits and their meaning.

Windows XP allows applications to allocate **large pages.** A large page is a set of contiguous pages that the operating system treats as one page. Large pages are useful when the system knows it will repeatedly need the same large chunk of code or data. The system needs to store only one entry in the TLB. Because the page is bigger, it is more likely to be accessed, and less likely to be deleted from the TLB. Page access is accelerated because the VMM can look up address translations in the TLB rather than consulting page tables.[204] Windows XP stores information that cannot be swapped to disk (i.e., the nonpaged pool) and memory maps on large pages.[205]

Page state bit	***Definition***
Valid	PTE is valid—it points to a page of data.
Modified	Page in memory is no longer consistent with the version on disk.
Transition	VMM is in the process of moving the page to or from disk. Pages in transition are always invalid.

Figure 21.6 | *Page states.*[206]

Windows XP imposes several restrictions on the use of large pages:

- Each type of processor has a minimum size for large pages, usually 2MB or greater. The size of every large page must be a multiple of that minimum large page size.[207]
- Large pages always allow read and write access. This means that read-only data, such as program code and system DLLs, cannot be stored in large pages.[208]
- The pages that compose a large page must be contiguous in both virtual and physical memory.[209, 210]

Windows XP uses copy-on-write pages (see Section 10.4.6, Sharing in a Paging System) as one of its lazy allocation mechanisms. The system manages copy-on-write pages using **prototype page tables.** The system also uses prototype page tables to manage file mappings. A file mapping is a portion of main memory that multiple processes may access simultaneously to communicate with one another (this is discussed further in Section 21.10.3, Shared Memory). The PTE of a copy-on-write page does not point directly to the frame in which the page is stored. Instead, it points to a **Prototype Page Table Entry (PPTE),** a 32-bit record that points to the location of the shared page.

When a process modifies a page whose PTE protection bits indicate that it is a copy-on-write page, the VMM copies the page to a new frame and sets the process's PPTE to reference the new location. All of the other processes's PPTE's continue pointing to the original frame. Using copy-on-write pages conserves memory, because processes share main memory page frames. As a result, each process can store more of its pages in main memory. This causes fewer page faults, because there is more space in main memory, and once the VMM fetches a page from disk, it is less likely to move the page back to disk. However, PPTEs add a level of indirection; it takes four memory references instead of three to translate an address. Therefore, translating an address that is not in the TLB is more expensive for copy-on-write pages than for normal pages.[211]

21.7.2 Memory Allocation

Windows XP performs memory allocation in three stages (Fig. 21.7). A process must first **reserve** space in its virtual address space. Processes can either specify what virtual addresses to reserve or allow the VMM to decide. A process may not access space it has reserved until it **commits** the space. When a process commits a page, the VMM creates a PTE and ensures that there is enough space in either main memory or a pagefile for the page. A process usually commits a page only when it is ready to write to the page; however, the Windows API supports single-step reserve and commit. Separating memory allocation into two steps allows a process to reserve a large contiguous virtual memory region and sparsely commit portions of it.[212, 213] Finally, when a process is ready to use its committed memory, it accesses the committed virtual memory. At that point, the VMM writes the data to a zeroed

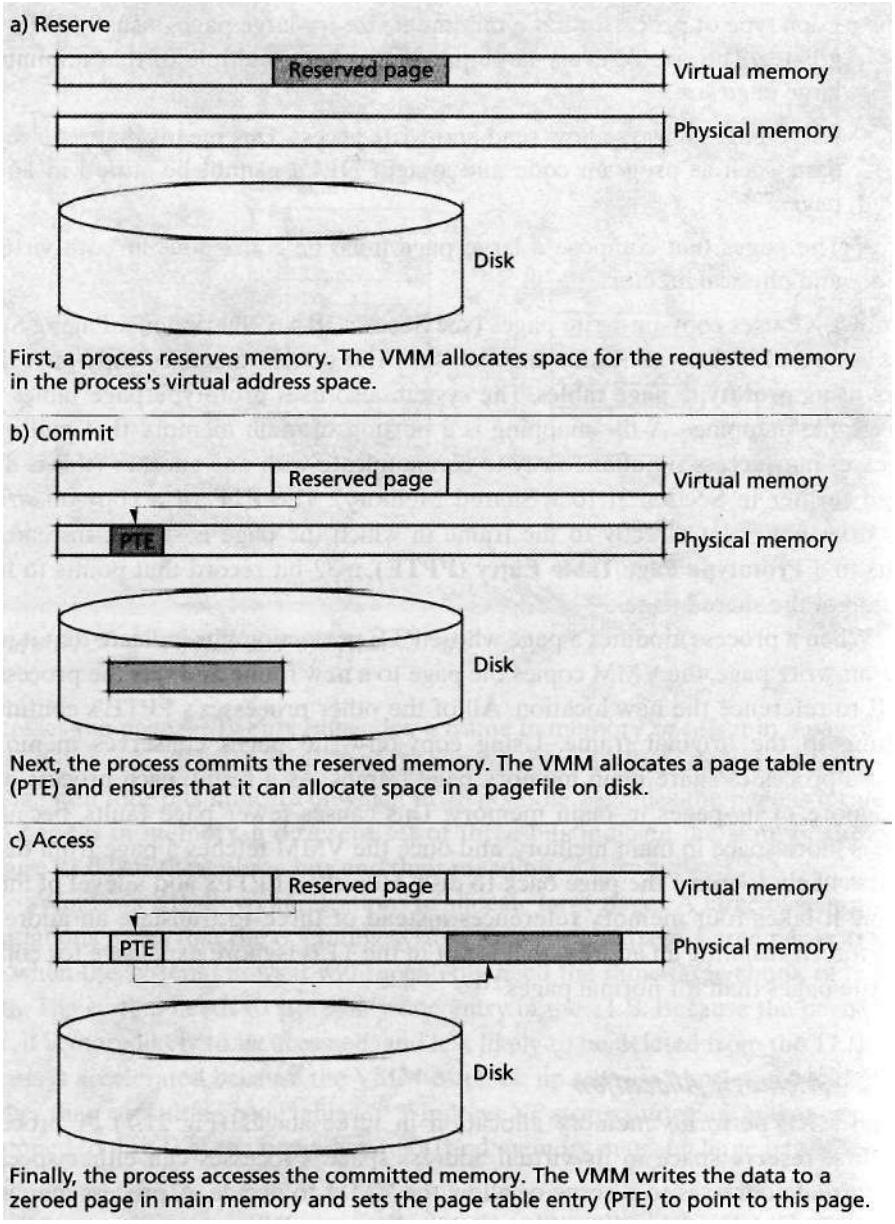


Figure 21.7 | Memory allocation stages.

page in main memory. This three-stage process ensures that processes use only as much space in main memory as they need, rather than the amount they reserve.[214]

A system rarely has enough main memory to satisfy all processes. Previous versions of Windows allowed an application that required more main memory to

function properly to issue **must-succeed requests.** Device drivers often issued these requests, and the VMM always fulfilled a must-succeed request. This led to system crashes when the VMM was forced to allocate main memory when none was available. Windows XP does not suffer from this problem; it denies all must-succeed requests. The system expects components to handle a denied memory allocation request without crashing.[215, 216]

Windows XP has a special mechanism for handling low-memory conditions when all or most of main memory is allocated. Under normal circumstances, Windows XP optimizes performance by handling multiple memory allocation requests simultaneously. When there are few page frames in main memory to allocate, the system runs into the same scarce-resource problem faced by all multiprogramming systems. Windows XP solves this problem via a process called **I/O throttling;** when the system detects that it has only a few available page frames, it begins to manage memory one page at a time. I/O throttling slows the system, because the VMM ceases to manage multiple requests in parallel but instead retrieves pages from disk only one at a time. However, it does make the system more robust and helps prevent crashes.[217]

To track main memory, Windows XP uses a **page frame database,** which lists the state of each frame, ordered by page frame number. There are eight possible states, as shown in Fig. 21.8.

The system tracks page frames by state. It has a singly linked list, called a **page list,** for each of the eight states. A page list is referred to by the state of its pages; for example, the Free Page List contains all free pages, the Zeroed Page List contains all zeroed pages and so on. The page lists permit the VMM to swap pages and allocate memory quickly. For example, to allocate two new pages, the VMM

Frame State	***Definition***
Valid	Page belongs to a process's working set and its PTE is set to valid.
Transition	Page is in the process of being transferred to or from disk.
Standby	Page has just been removed from a process's working set; its PTE is set to invalid and in transition.
Modified	Page has just been removed from a process's working set; it is not consistent with the on-disk version. The VMM must write this page to disk before freeing this page. The PTE of this page is set to invalid and in transition.
Modified No-Write	Page has just been removed from a process's working set; it is not consistent with the on-disk version. The VMM must write an entry to the log file before freeing this page. The PTE of this page is set to invalid and in transition.
Free	Page frame does not contain a valid page; however it might contain an invalid page that has no PTE and is not part of any working set.

Figure 21.8 | Page frame states.[218] *(Part 1 of 2.)*

Frame State	***Definition***
Zeroed	Page frame is not part of any working set and all of its bits have been set to zero. For security reasons, only zeroed page frames are allocated to processes.
Bad	Page frame has generated a hardware error and should not be used.

Figure 21.8 | Page frame states[218] *(Part 2 of 2.)*

needs to access only the first two frames in the Zeroed Page List. If the Zeroed Page List is empty, the VMM takes a page from another list, using the algorithm described in Section 21.7.3, Page Replacement.[219]

The system uses **Virtual Address Descriptors (VADs)** to manage the virtual address space of each process. Each VAD describes a range of virtual addresses allocated to the process.[220]

Windows XP attempts to anticipate requests for pages on disk and move pages to main memory to prevent page faults. A process can invoke API functions to tell the system the process's future memory requirements.[221] In general, the system employs a policy of demand paging, loading pages into memory only when a process requests them. It also loads several nearby pages—spatial locality implies. that these pages are likely to be referenced soon. The Windows file systems divide disk space into **clusters** of bytes; pages in the same cluster are, by definition, part of the same file. Windows XP takes advantage of spatial locality by loading all pages in the same cluster at once, a paging policy referred to as **clustered demand paging.** These two mechanisms reduce disk seek time because only one disk seek is needed to fetch the entire cluster. Prefetching, however, potentially places some unneeded pages into main memory, leaving less space for needed pages and increasing the number of page faults. It might also decrease efficiency during periods of heavy disk I/O by forcing the system to page out pages that are needed right away for pages that are not needed yet.[222]

Windows XP reduces the time needed to load (i.e., start) applications, including the operating system itself, by prefetching files. The system records what pages are accessed and in what order during the last eight application loads. Before loading an application, the system asks for all of the pages it will need in one asynchronous request, thus decreasing disk seek time.[223, 224]

The system decreases the time it takes to load Windows by simultaneously prefetching pages and initializing devices. When Windows XP starts, it must direct numerous peripheral hardware devices to initialize—the system cannot continue loading until they are done. This is the ideal time to perform prefetching, because the system is otherwise idle.[225]

Windows XP uses the **Logical Prefetcher** to perform prefetching. The user can set a special registry key to tell Windows XP for which scenarios, such as all application loads or Windows start-up, the user wants the system to run the Logical

Prefetches[226] When an application loads, Windows XP stores a trace of memory accesses in a scenario file. Each application, including the Windows operating system, has its own scenario file. The system updates scenario files ten seconds after a successful application load. Storing scenario files has a price: recording each memory access uses processor cycles and occupies space on the disk. However, the reduced disk I/O offsets the extra time it takes to store the memory trace, and most users are willing to give up some disk space in return for significantly faster load time for all applications, including for Windows itself.[227]

The Logical Prefetcher accesses pages based on their location on disk rather than waiting for a process to explicitly request them. As a result, Windows XP can load applications faster and more efficiently. However, the pages and files indicated in scenario files could be scattered throughout the disk. To further optimize application load time, the Logical Prefetcher periodically reorganizes portions of the hard drive to ensure that all files that the system has previously prefetched are contiguous on disk. Every few days, when the system is idle, the Logical Prefetcher updates a layout file which contains the ideal disk layout and uses this layout to rearrange pages on disk.[228]

21.7.3 Page Restatement

Windows XP bases its page-replacement policy on the working set model. Recall from Section 11.7, Working Set Model, that a process's working set is the set of pages that a process is currently using. Because this is difficult to determine, Windows XP simply considers a process's **working set** to be all of its pages in main memory Pages not in the process's working set, but mapped to its virtual address space, are stored on disks in pagefiles.[229, 230] Storing pagefiles on different physical disks accelerates swapping because it enables the system to read and write multiple pages concurrently[231]

When memory becomes scarce, the **balance set manager** moves parts of different processes' working sets to pagefiles. Windows XP employs a **localized least-recently used** (LRU) policy to determine which pages to move to disk. The policy is localized by process. When the system needs to free a frame to meet a process's request, it removes a page belonging to the process which the process has not recently accessed. The VMM assigns each process a **working set maximum** and a **working set minimum** that denote an acceptable range for the size of the process's working set. Whenever a process whose working set size is equal to its working set maximum requests an additional page, the balance set manager moves one of the process's pages to secondary storage, then fulfills the request. A process using less than its working set minimum is not in danger of losing pages unless available memory drops below a certain threshold. The balance set manager checks the amount of free space in memory once per second and adjusts the working set maximums accordingly. If the number of available bytes exceeds a certain threshold, the working set maximums of processes working at their working set maximum shift upward; if it drops below a certain threshold, all working set maximums shift downward.[232]

Windows XP does not wait until main memory is full to move pages to disk. It regularly tests each process to ensure that only the pages the process needs are in main memory. Windows XP uses **page trimming** to remove pages from memory. Periodically, the balance set manager moves all of a process's pages above the working set minimum from the Valid Page List to the Standby Page List, Modified Page List, or Modified No-Write Page List, as appropriate. Collectively, these are called the **transitional page list,** although there is no such actual list in memory.[233]

The VMM chooses which pages to trim, using the localized LRU policy, as illustrated in Fig. 21.9. The system sets the status bits of the PTEs of trimmed pages to invalid, meaning the PTEs no longer point to a valid frame, but the system does not modify the PTEs in any other way. The state in the page frame database of trimmed pages is set to transition, standby, modified or modified no-write.[234] If the process requests a page that is in one of these transitional page lists, the MMU issues a **transitional fault.** In this situation, the VMM removes the page from the transitional list and puts it back on the Valid Page List. Otherwise, the page is reclaimed. A system thread writes pages in the Modified Page List to disk. Components, such as file systems, use the Modified No-Write Page List to ensure that the VMM does not write a dirty page to disk without first making an entry in a log file. The application notifies the VMM when it is safe to write the page.[235] Once a modi-

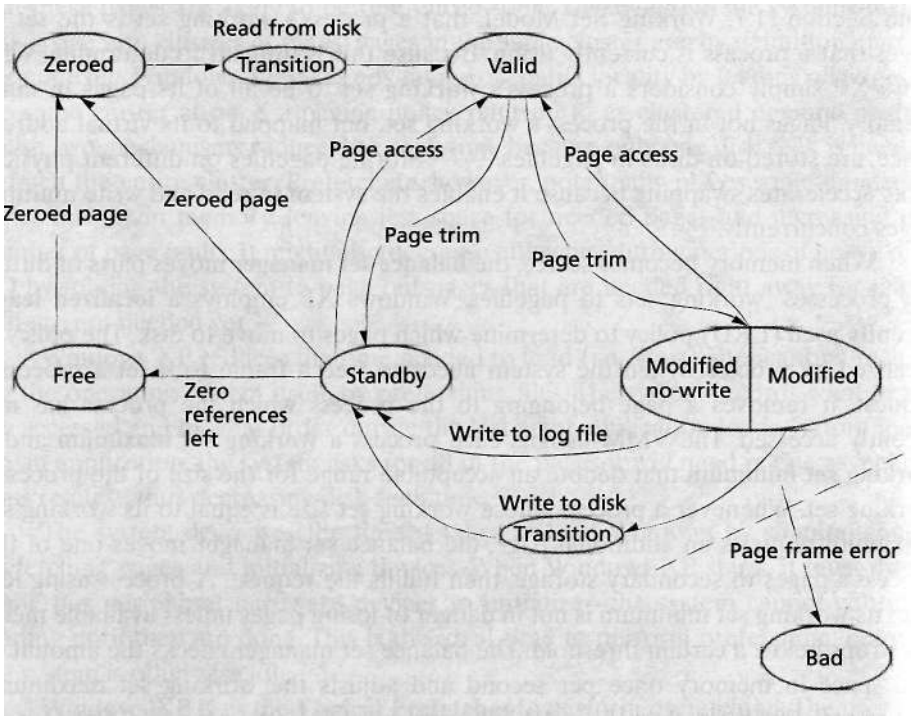


Figure 21.9 | Page-replacement process.

fled or modified no-write page becomes consistent with the data on disk, the VMM moves the page to the Standby Page List. After a waiting period, the VMM moves pages on the Standby Page List to the Free Page List. Later, another low-priority thread zeroes the bits of pages on the Free Page List and moves the pages to the Zeroed Page List. Windows XP zeroes pages to prevent a process that is allocated a page from reading the (potentially sensitive) information that another process previously stored on that page. When the system needs to allocate a new page to a process, it claims a page from the Zeroed Page List.[236]

The localized LRU algorithm requires certain hardware features not available on all systems. Because portability between different platforms is an important design criterion for all Windows operating systems, many early Windows systems avoided using hardware-specific features to ensure that the same operating system could be used on multiple platforms. Operating systems commonly simulate LRU page replacement with the clock algorithm (see Section 11.6.7, Modifications to FIFO: Second-Chance and Clock Page Replacement). Doing so requires an accessed bit which the system sets on for each page that has been accessed within a certain time interval. Many early computers did not have an accessed bit. As a result, early versions of Windows NT used the FIFO page-replacement policy (see Section 11.6.2, First-In-First-Out (FIFO) Page Replacement) or even an effectively random page replacement. Microsoft later developed versions of Windows specifically for Intel IA-32 and other platforms that support the accessed bit, allowing Windows XP to employ the more effective LRU algorithm.[237]

Some data cannot be paged out of memory. For example, code that handles interrupts must stay in main memory, because processing a page fault during an interrupt would cause an unacceptable delay and might even crash the system. Paging passwords presents a security risk if the system crashes, because an unencrypted copy of secret data would be stored on disk. All such data is stored in a designated area of system space called the **nonpaged pool.** Data that can be sent to disk is stored in the **paged pool.**[238] Portions of device drivers and VMM code are stored in the nonpaged pool.[239]

A device driver developer must consider several trade-offs when deciding what portions of the code to place in the nonpaged pool. Space in the nonpaged pool is limited. In addition, code in the nonpaged pool may not access paged pool code or data because the VMM might be forced to access the disk, which defeats the purpose of nonpaged pool code.[240]

21.8 File Systems Management

Windows XP file systems consist of three layers of drivers. At the very bottom are various volume drivers that control a specific hardware device, such as the hard disk. File system drivers, which compose the next level, implement a particular file system format, such as New Technology File System (NTFS) or File Allocation Table (FAT). These drivers implement what a typical user views as the file system: a

hierarchical organization of files and the related functions to manipulate these files. Finally, file system filter drivers accomplish high-level tasks such as virus protection, compression and encryption.[241][242] NTFS is the native Windows XP file system (and the one described in depth in this case study), but FAT16 and FAT32 are also supported (see Section 13.6.3, Tabular Noncontiguous File Allocation) and typically used for floppy disks. Windows XP also supports Compact Disc File System (CDFS) for CDs and Universal Disk Format (UDF) for CDs and DVDs. The following subsections introduce Windows XP file system drivers and describe the NTFS file system.[243]

21.8.1 File System Drivers

File system drivers cooperate with the I/O manager to fulfill file I/O requests. File system drivers provide a link between the logical representation of the file system exposed to applications and its corresponding physical representation on a storage volume. The I/O manager interface enables Windows XP to support multiple file systems. Windows XP divides each data storage device into one or more volumes, and associates each volume with a file system.

To understand how file system drivers and the I/O manger interact, consider the flow of a typical file I/O request. A user-mode thread that wishes to read data from a file sends an I/O request via a subsystem API call. The I/O manager translates the thread's file handle into a pointer to the file object and passes the pointer to the appropriate file system driver. The file system driver uses the pointer to locate the file object and determine the location of the file on disk. Next, the file system driver passes the read request through the layers of drivers and eventually the request reaches the disk. The disk processes the read request and returns the requested data.[244]

A file system driver can be either a local file system driver or a remote file system driver. Local file system drivers process I/O for hardware devices such as hard disk drives or DVD drives. The preceding paragraph described the role of a local file system driver in fulfilling a disk I/O request. Remote file system drivers transfer files to and from remote file servers via network protocols. Remote file system drivers cooperate with the I/O manager, but instead of interacting with volume drivers, remote file system drivers interact with remote file system drivers on other computers.[245]

In general, a file system driver can act as a black box; it provides support for a particular file system, such as NTFS, independent of the underlying storage volume on which the files reside. File system drivers provide the link between the user's view of a file system and the data's actual representation on a storage volume.

21.8.2 NTFS

NTFS is the default file system for Windows XP. When creating the NT line of operating systems, Microsoft decided to construct a new file system to address the limitations of the FAT file system used in DOS and older versions of Windows. FAT.

which uses a tabular noncontiguous allocation scheme (see Section 13.6.3, Tabular Noncontiguous File Allocation), does not scale well to large disk drives. For example, a file allocation table for FAT32 (the file system included in Windows ME) on a 32GB hard drive using 2KB clusters consumes 64MB of memory. Furthermore, FAT32 can address no more than 2^{32} data blocks. FAT's addressing limitation was a problem for FAT12 (12-bit FAT) and FAT16 (16-bit FAT) and undoubtedly will be a problem in the future for FAT32. To avoid these shortcomings, NTFS uses an indexed approach (see Section 13.6.4, Indexed Noncontiguous File Allocation) with 64-bit pointers. This allows NTFS to address up to 16 exabytes (i.e., 16 billion gigabytes) of storage. Furthermore, NTFS includes additional features that make a file system more robust. These features include file compression, file encryption, support for multiple data streams and user-level enhancements (e.g., support for hard links and easy file system and directory browsing).

Master File Table

The most important file on an NTFS volume is the **Master File Table (MFT).** The MFT stores information about all files on the volume, including file metadata (e.g., time of creation, the filename and whether the file is read-only, archive, etc.). The MFT is divided into fixed-size records, usually 1KB long. Each file has an entry in the MFT, which consists of at least one record, plus additional ones if necessary.[246]

NTFS stores all information about a file in attributes, each consisting of a header and data. The header contains the attribute's type (e.g., f i 1 e_name), name (necessary for files with more than one attribute of the same type) and flags. If any of a file's attributes do not fit into the file's first MFT record, NTFS creates a special attribute that stores pointers to the headers of all attributes located on different records. An attribute's data can be of variable length. Attribute data that fits in an MFT entry is stored in the entry for quick access; attributes whose data reside in the MFT entry are called **resident attributes.** Because the actual file data is an attribute, the system stores small files entirely within their MFT entry. NTFS stores **nonresident attributes,** the attributes whose data does not fit inside the MFT entry, elsewhere on disk.[247]

The system records the location of nonresident attributes using three numbers. The logical cluster number (LCN) tells the system on which cluster on disk the attribute is located. The run length indicates how many clusters the attribute spans. Because a file attribute might be split into multiple fragments, the virtual cluster number (VCN) tells the system to which cluster in the attribute the LCN points. The first VCN is zero.[248]

Figure 21.10 shows a sample multirecord MFT entry that contains both resident and nonresident attributes. Attribute 3 is a nonresident attribute. The data portion of attribute 3 contains a list of pointers to the file on disk. The first 10 clusters are stored on disk clusters (i.e., LCNs) 1023-1032. The eleventh cluster, VCN 10, is stored on disk cluster 624. Attributes 1 and 2 are resident attributes, so they stored on the first MFT record; attributes 3 and 4 are nonresident attributes stored on the second MFT record.

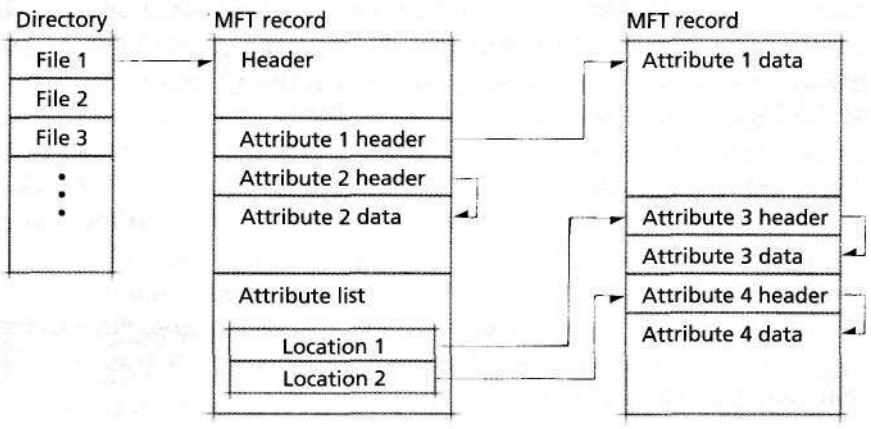


Figure 21.10 | *Master File Table (MFT) entry for a sample file.*

NTFS stores directories as files. Each directory file contains an attribute called index that stores the list of files inside the directory. Each file's directory entry contains the file's name and standard file information, such as the time of the most recent modification to the file and the file's access (e.g., read-only). If the index attribute is too large to be a resident attribute, NTFS creates index buffers to contain the additional entries. An additional attribute, the index_root, describes where on disk any index buffers are stored.

For easy directory searching, NTFS sorts directory entries alphabetically. NTFS structures a large directory (one that is not a resident attribute) in a two-level B-tree. Figure 21.11 shows a sample directory. Suppose the system directs NTFS to find the file e.txt. NTFS begins searching in the index attribute, where it finds a.txt. The file a.txt comes before e.txt, so NTFS scans to the next file, j.txt. Because j.txt comes after e.txt, NTFS proceeds to the appropriate child node of a.txt. NTFS searches in the first index buffer, obtaining its location on the volume from information in the index_root attribute. This index buffer begins with b.txt; NTFS scans the first index buffer until it finds e.txt.[249]

Windows XP allows multiple directory entries (either in the same directory or in different directories) to point to the same physical file. The system can create a new path (i.e., a new directory entry) to an already existing file, i.e., a hard link (see Section 13.4.2, Metadata). It first updates the existing file's MFT entry by adding a new file_name attribute and incrementing the value of the hard_link attribute. Next, it creates the new directory entry to point to the MFT entry of the existing file. NTFS treats each newly created hard link to a file exactly as it treats the original file name. If a user "moves" the file, the value of a file_name attribute is changed from the old to the new path name. NTFS does not delete the file until all hard links to the file are removed. It does not permit hard links to directory files.

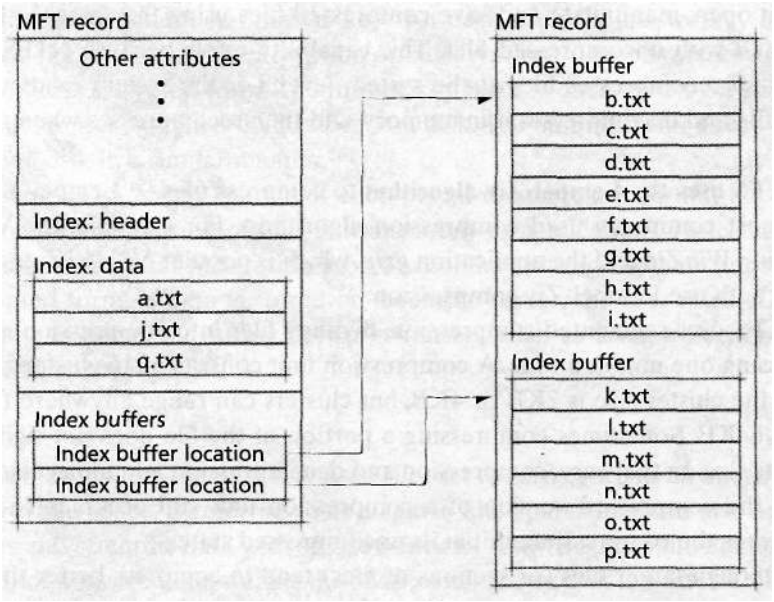


Figure 21.11 | Directory contents are stored in B-trees.

and because hard links point directly to MFT entries, a hard link must point to a file residing on the same physical volume.[250, 251]

Data Streams

The contents (i.e., the actual file data) of an NTFS file are stored in one or more **data streams.** A data stream is simply an attribute in an NTFS file that contains file data. Unlike many other file systems, including FAT, NTFS allows users to place multiple data streams into one file. The unnamed **default data stream** is what most people consider to be the "contents" of a file. NTFS files can have multiple **alternate data streams.** These streams can store metadata about a file, such as author, summary or version number. They also can be a form of supplemental data, such as a small preview image associated with a large bitmap or a backup of a text file.

NTFS considers each data stream of a file to be an attribute of type Data. Recall that header information for an attribute stores both the attribute type (in this case Data) and the attribute's name. NTFS differentiates between different alternate data streams using the attribute's name; the default data stream is unnamed. Using separate attributes for each data stream enables the system to access any data stream without scanning through the rest of the file.[252]

File Compression

NTFS permits users to compress files and folders through the GUI interface or via system calls. When a user compresses a folder, the system compresses any file or

folder that is added to the compressed folder. Compression is transparent; applications can open, manipulate and save compressed files using the same API calls as they would with uncompressed files. This capability exists because NTFS decompresses and recompresses files at the system level; i.e., the system reads the compressed file and decompresses it in memory, and then recompresses when the file is saved.[253]

NTFS uses the **Lempel-Ziv algorithm** to compress files.[254] Lempel-Ziv is one of the most commonly used compression algorithms. For example, the Windows application *WinZip* and the application *gzip,* which is popular on UNIX-compatible systems, both use Lempel-Ziv compression.[255]

NTFS uses segmented compression, dividing files into **compression units** and compressing one unit at a time. A compression unit consists of 16 clusters; on most systems the cluster size is 2KB or 4KB, but clusters can range anywhere from 512 bytes to 64KB. Sometimes compressing a portion of the file does not significantly reduce its size. In that case, compression and decompression add unnecessary overhead. If the compressed version of a compression unit still occupies 16 clusters, NTFS stores the compression unit in its uncompressed state.[256]

Although larger files (or sections of files) tend to compress better than small files, this segmented approach decreases file access time. Segmented compression enables applications to perform random file I/O without decompressing the entire file. Segmented compression also enables NTFS to compress a file while an application modifies it. Modified portions of the file are cached in memory. Because a portion of the file that has been modified once is likely to be modified again, constantly recompressing the file while it is open is inefficient. A low-priority lazy-writer thread is responsible for compressing the modified data and writing it to disk.[257]

File Encryption

NTFS performs file encryption, like file compression, by dividing the file into units of 16 clusters and encrypting each unit separately. Applications can specify that a file should be encrypted on disk when created, or if the user has sufficient access rights, an application executing on the user's behalf can encrypt an existing file. The system will encrypt any file or folder for a user-mode thread, except system files, system folders and root directories. Because these files and folders are shared among all users, no user-mode thread can encrypt them (and therefore not allow other users access to them).[258]

NTFS uses a public/private key pair (see Section 19.2, Cryptography) to encrypt files. NTFS creates **recovery keys** that administrators can use to decrypt any file. This prevents files from being permanently lost when users forget their private keys or change jobs.[259]

NTFS stores encryption keys in the nonpaged pool in system memory. This is done for security reasons to ensure that the encryption keys are never written to disk. Storing keys on disk would compromise the security of the storage volume, because malicious users could first access the encryption keys, then use them to

decrypt the data.[260] Instead of storing the actual keys, NTFS encrypts private keys using a randomly generated "master key" for each user; the master key is a symmetric key (i.e., it does both encryption and decryption). The private key is stored on disk in this encrypted form. The master key itself is encrypted by a key generated from the user's password and stored on disk. The administrator's recovery key is stored on disk in a similar manner.[261]

When NTFS encrypts a file, it encrypts all data streams of that file. NTFS's encryption and decryption of files is transparent to applications. Note that NTFS encryption ensures that files residing on a secondary storage volume are stored in an encrypted form. If data is stored on a remote server, the user must take extra precautions to protect it during network transfer, such as using a Secure Sockets Layer (SSL) connection.[262]

Sparse Files

A file in which most of the data is unused (i.e., set to zero), such as an image file that is mostly white space, is referred to as a sparse file. Sparse files can also appear in databases and scientific data with sparse matrices. A 4MB sparse file might contain only 1MB of nonzero data; storing the 3MB of zeroes on disk is wasteful. Compression, although an option, degrades performance, owing to the need to decompress and compress the files when they are used. Therefore, NTFS provides support for sparse files.[263]

A thread explicitly specifies that a file is sparse. If a thread converts a normal file to a sparse file, it is responsible for indicating areas of the file that contain large runs of zeroes. NTFS does not store these zeroes on disk but keeps track of them in a zero block list for the file. Each entry in the list contains the start and end position of a run of zeroes in the file. The used portion of the file is stored in the usual manner on disk. Applications can access just the used file segments or the entire file, in which case NTFS generates streams of zeroes where necessary.[264]

When an application writes new data to a sparse file, the writing thread is responsible for indicating areas of the data that should not be stored on disk but rather in the zero block list. If a write contains an entire compression unit of zeroes (i.e., 16 clusters of zeroes), NTFS recognizes this and records the empty block in the zero block list instead of writing the compression unit full of zeroes to disk. (The write operation must be done using a special API call that notifies NTFS that the written data consists exclusively of zeroes.) As a result, while an application can specify exact ranges of zeroes for better memory management, the system also performs some memory optimizations on its own.[265]

Resparse Points and Mounted Volumes

Recall our earlier discussion of NTFS hard links from the Master File Table subsection. We noted that they are limited because they cannot point to directories, and the data to which they point must reside on the same volume as the hard link. NTFS includes a file attribute, called a **reparse point,** to address these limitations. A rep-

arse point contains a 32-bit tag and can contain up to 16KB of attribute data. When the system accesses a file with a reparse point, the system first processes the information in the reparse point. NTFS uses the tag to determine which file system filter driver should handle the data in the reparse point. The appropriate filter driver reads the attribute data and performs some function, such as scanning for viruses, decrypting a file or locating the file's data.[266]

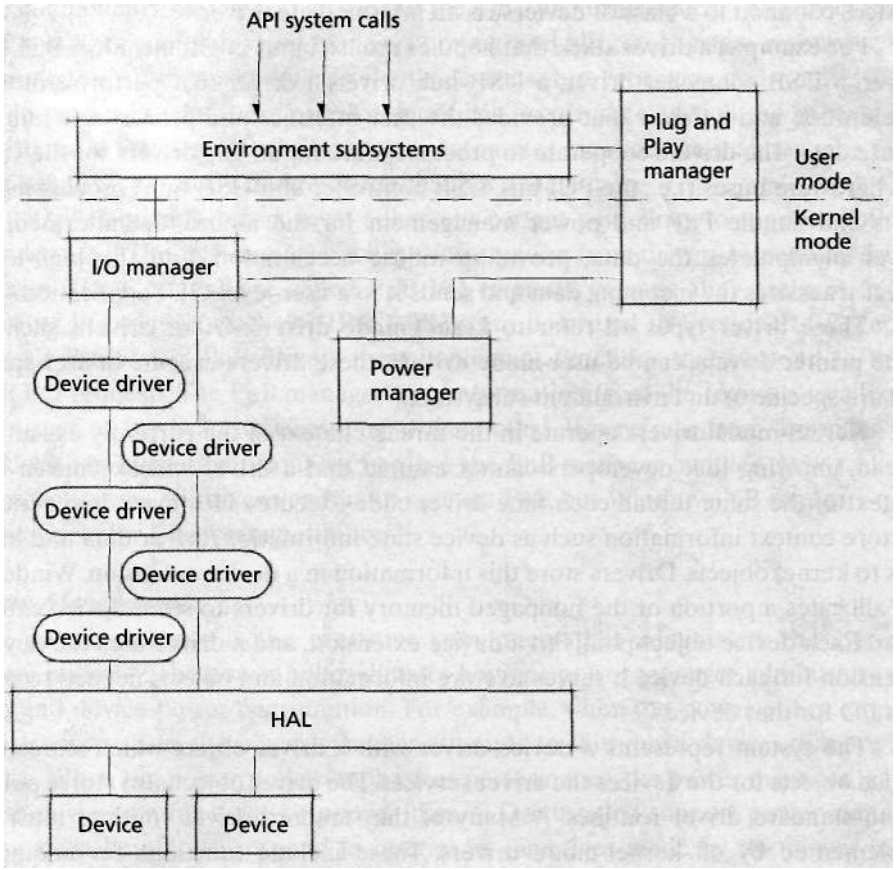
Reparse points permit applications to establish links to files on another volume. For example, little-used hard disk data is sometimes moved to a tertiary storage device such as a tape drive. Although NTFS might delete the file's data after it is copied to the tertiary storage (to save space on the hard disk), it retains the file's MFT record. NTFS also adds a reparse point specifying from where to retrieve the data. When an application accesses the data, NTFS encounters the reparse point and uses its tag to call the appropriate file system driver, which retrieves the data from tertian storage and copies it back to disk. This operation is transparent to the application.[267]

Additionally, reparse points are used to mount volumes. The reparse point data specifies the root directory of the volume to mount and how to find the volume, and the file system driver uses this information to mount that volume. In this way, a user can browse a single directory structure that includes multiple volumes. A reparse point can associate any directory within an NTFS volume with the root directory of any volume; this directory is called a mounting directory. NTFS redirects all access to the mounting directory to the mounted volume.[268]

A reparse point can be used to create a **directory junction**—a directory referring to another directory, similar to a symbolic directory link in Linux. The reparse point specifies the pathname of the directory to which the directory junction refers. This is similar to mounting volumes, except that both directories must be within the same volume. The referring directory must be empty; a user must remove the directory junction before inserting files or folders into the referring directory.[269]

21.9 Input/Output Management

Managing input/output (I/O) in Windows XP involves many operating system components (Fig. 21.12). User-mode processes interact with an environment subsystem (such as the Win32 subsystem) and not directly with kernel-mode components. The environment subsystems pass I/O requests to the **I/O manager,** which interacts with device drivers to handle such requests. Often, several device drivers, organized into a driver stack, cooperate to fulfill an I/O request.[270] The **Plug and Play (PnP) manager** dynamically recognizes when new devices are added to the system (as long as these devices support PnP) and allocates and deallocates resources, such as I/O ports or DMA channels, to them. Most recently developed devices support PnP. The **power manager** administers the operating system's power management policy. The power policy determines whether to power down devices to conserve energy or keep them fully powered for high responsiveness.[271] We describe the PnP manager and power manager in more detail later in this section. First, we describe how these components and device drivers cooperate to manage I/O in Windows XP.



21.9.1 Device Drivers

Windows XP stores information about each device in one or more **device objects.** A device object stores device-specific information (e.g., the type of device and the current I/O request being processed) and is used for processing the device's I/O requests. Often, a particular device has several device objects associated with it, because several drivers, organized into a **driver stack,** handle I/O requests for that device. Each driver creates a device object for that device.[272]

The driver stack consists of several drivers that perform different tasks for I/O management. A **low-level driver**—a driver that interacts most closely with the HAL—controls a peripheral device and does not depend on any lower-level drivers; a bus driver is a low-level driver. A **high-level driver** abstracts hardware specifics and passes I/O requests to low-level drivers. An NTFS driver is a high-level driver that abstracts how data is stored on disk. An **intermediate driver** can be interposed between high- and low-level drivers to filter or process I/O requests and

export an interface for a specific device; a class driver (i.e., a driver that implements services common to a class of devices) is an intermediate driver.

For example, a driver stack that handles mouse input might include a PCI bus driver, a USB controller driver, a USB hub driver, a driver that performs mouse acceleration and a driver that provides the user interface and processes incoming mouse data. The drivers cooperate to process mouse input. The drivers for the various hardware buses (i.e., the PCI bus, USB controller and USB hub) process interrupts and handle PnP and power management for the mouse; the intermediate driver manipulates the data, providing mouse acceleration; and the high-level driver translates the incoming data and sends it to a user-level GUI application.[273]

These driver types all refer to **kernel-mode drivers.** Other drivers, such as some printer drivers, can be **user-mode drivers;** these drivers execute in user space and are specific to an environment subsystem.[274]

Kernel-mode drivers operate in the thread context of the currently executing thread, implying that developers cannot assume that a driver will execute in the context of the same thread each time driver code executes. However, drivers need to store context information such as device state information, driver data and handles to kernel objects. Drivers store this information in a **device extension.** Windows XP allocates a portion of the nonpaged memory for drivers to store device extensions. Each device object points to a device extension, and a driver uses the device extension for each device it serves to store information and objects needed to process I/O for that device.[275]

The system represents a device driver with a **driver object** which stores the device objects for the devices the driver services. The driver object also stores pointers to standard driver routines.[276] Many of the standard driver routines must be implemented by all kernel-mode drivers. These include functions for adding a device, unloading the driver and certain routines for processing I/O requests (such as read and write). Other standard driver routines are implemented only by certain classes of drivers. For example, only low-level drivers need to implement the standard routine for handling interrupts. In this way, all drivers expose a uniform interface to the system by implementing a subset of the standard driver routines.[277]

Plug and Play (PnP)

Plug and Play (PnP), introduced in Section 2.4.4, describes the ability of a system to dynamically add or remove hardware components and redistribute resources (e.g.. I/O ports or DMA channels) appropriately. A combination of hardware and software support enables this functionality. Hardware support involves recognizing when a user adds or removes components from a system and easily identifying these components. Both the operating system and third-party drivers must cooperate to provide software support for PnP. Windows XP support for PnP involves utilizing information provided by hardware devices, the ability to dynamically allocate resources to devices and a programming interface for device drivers to interact with the PnP system.[278]

Windows XP implements PnP with the **PnP manager,** which is divided into two components, one in user space and the other in kernel space. The kernel-mode PnP manager configures and manages devices and allocates system resources. The user-mode PnP manager interacts with device setup programs and notifies user-mode processes when some device event has occurred for which the process has registered a listener. The PnP manager automates most device maintenance tasks and adapts to hardware and resource changes dynamically.[279]

Driver manufacturers must adhere to certain guidelines to support PnP for Windows XP. The PnP manager collects information from drivers by sending driver queries. These queries are called **PnP I/O requests,** because they are sent in the form of I/O request packets (IRPs). IRPs are described in Section 21.9.2, Input/ Output Processing. PnP drivers must implement functions that respond to these PnP I/O requests. The PnP manager uses information from the requests to allocate resources to hardware devices. PnP drivers must also refrain from searching for hardware or allocating resources, because the PnP manager handles these tasks.[280] Microsoft urges all driver vendors to support PnP, but Windows XP supports non-PnP drivers for legacy compatibility.[281]

Power Maanagement

The **power manager** is the executive component that administers the system's **power policy.**[282] The power policy dictates how the power manager administers system and device power consumption. For example, when the power policy emphasizes conservation, the power manager attempts to shut down devices that are not in use. When the power policy emphasizes performance, the power manager leaves these devices on for quick response times.[283] Drivers that support power management must be able to respond to queries or directives made by the power manager.[284] The power manager queries a driver to determine whether a change in the power state of a device is feasible or will disrupt the device's work. The power manager also can direct a driver to change the power state of a device.[285]

Devices have power states DO, D1, D2 and D3. A device in state DO is fully powered, and a device in D3 is off.[286] In states Dl and D2, the device is in a sleep (i.e., low-powered) state, and when the device returns to the DO state, the system will need to reinitialize some of its context. A device in Dl retains more of its context and is in a higher-powered state than a device in D2.[287] In addition to controlling devices' power states, the power manager controls the overall system's power state, which is denoted by values S0-S5. State SO is the fully powered working state, and S5 denotes that the computer is off. States S1-S4 represent states in which the computer is on, but in a sleeping state; a system in S1 is closest to fully powered, and in S4 is closest to off.[288]

Windows Driver Model (WDM)

Microsoft has defined a standard driver model—the **Windows Driver Model (WDM)**—to promote source-code compatibility across all Windows platforms.

Windows XP supports non-WDM drivers to maintain compatibility with legacy drivers, but Microsoft recommends that all new drivers be WDM drivers. This section briefly outlines the WDM guidelines.[289]

WDM defines three types of device drivers. **Bus drivers** interface with a hardware bus, such as a SCSI or PCI bus, provide some generic functions for the devices on the bus, enumerate these devices and handle PnP I/O requests. Each bus must have a bus driver, and Microsoft provides bus drivers for most buses.[290]

Filter drivers are optional and serve a variety of purposes. They can modify the behavior of hardware (e.g., provide mouse acceleration or enable a joystick to emulate a mouse) or add additional features to a device (e.g., implement security checks or merge audio data from two different applications for simultaneous playback). Additionally, filter drivers can sort I/O requests among several devices. For example, a filter driver might serve multiple storage devices and sort read and write requests between the devices. Filter drivers can be placed in numerous locations in the device stack—a filter driver that modifies hardware behavior is placed near the bus driver, whereas one that provides some high-level feature is placed near the user interface.[291]

A **function driver** implements a device's main function. A device's function driver does most of the I/O processing and provides the device's interface. Windows XP groups devices that perform similar functions into a device class (such as the printer device class).[292] A function driver can be implemented as a **class/miniclass driver pair.**[293] A class driver provides generic processing for a particular device class (e.g., a printer); a miniclass driver provides the functionality specific to a particular device (e.g., a particular printer model). Typically, Microsoft provides class drivers, and software vendors provide the miniclass drivers.[294]

All WDM drivers must be designed as either a bus driver, filter driver or function driver.[295] Additionally, WDM drivers must support PnP, power management and **Windows Management Instrumentation (WMI).**[296] Drivers that support WMI provide users with measurement and instrumentation data. This data includes configuration data, diagnostic data and custom data. Also, WMI drivers permit user applications to register for WMI driver-defined events. The main purpose of WMI is to provide hardware and system information to user processes and allow user processes greater control in configuring devices.[297]

21.9.2 Input/Output Processing

Windows XP describes I/O requests with **I/O request packets (IRPs).** IRPs contain all the information necessary to process an I/O request. An IRP (Fig. 21.13) consists of a header block and an I/O stack. Most header block information does not change during request processing. The header block records such information as the requestor's mode (either kernel or user), flags (e.g., whether to use caching) and information about the requestor's data buffer.[298] The header block also contains the **I/O status block,** which indicates whether an I/O request completed successfully or. if not, the request's error code.[299]

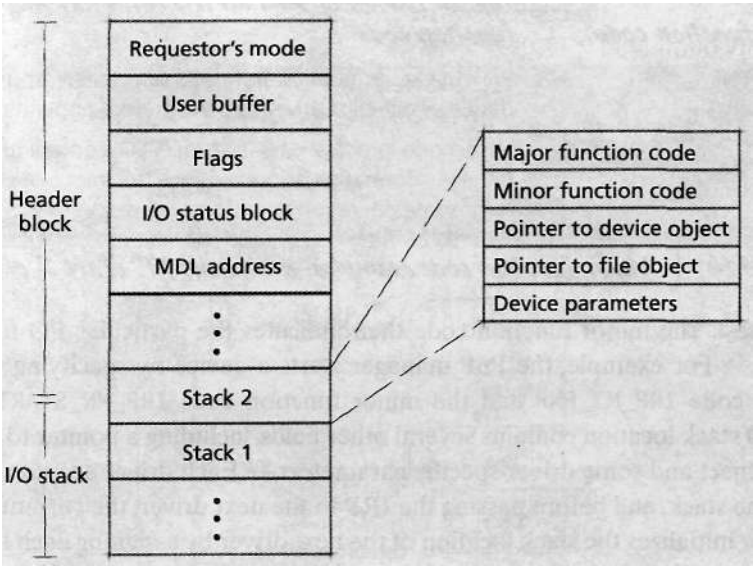


Figure 21.13 | I/O request packet (IRP).

The I/O stack portion of an IRP contains at least one different stack location for each driver in the device stack of the target device. Each I/O stack location contains specific information that each driver needs in order to process the IRP, in particular the IRP's **major function code** and the **minor function code.** Figure 21.14 lists several major function codes. The major function code specifies the general IRP function such as read (IRP_MJ_READ) or write (IRP_MJ_WRITE). In some cases, it designates a class of IRPs, such as IRP_MJ_PNP, which specifies that the IRP is a PnP

Major function code	***Typical reason to send an IRP with this major function code***
IRP_MJ_READ	User-mode process requests to read from a file.
IRP_MJ_WRITE	User-mode process requests to write to a file.
IRP_MJ_CREATE	User-mode process requests a handle to a file object.
IRP_MJ_CLOSE	All handles to a file object have been released and all outstanding I/O requests have been completed.
IRP_MJ_POWER	Power manager queries a driver or directs a driver to change the power state of a device.

***Figure 21.14** | Major function code examples in Windows XP. (Part 1 of 2.)*

Major function code	*Typical reason to send an IRP with this major function code*
IRP_MJ_PNP	PnP manager queries a driver, allocates resources to a device or directs a driver to perform some operation.
IRP_MJ_DEVICE_CONTROL	User-mode process calls a device I/O control function to retrieve information about a device or direct a device to perform some operation (e.g., format a disk).

Figure 21.14 | *Major function code examples in Windows XP. (Part 2 of 2.)*

I/O request. The minor function code then indicates the particular I/O function to perform.[300] For example, the PnP manager starts a device by specifying the major function code IRP_MD_PNP and the minor function code IRP_MN_START_DEVICE. Each I/O stack location contains several other fields, including a pointer to the target device object and some driver-specific parameters.[301] Each driver accesses one location in the stack, and before passing the IRP to the next driver, the currently executing driver initializes the stack location of the next driver by assigning each field in the stack a value.[302]

Processing an I/O Request

The I/O manager and device drivers cooperate to fulfill an I/O request. Figure 21.15 describes the path an IRP travels as the system fulfills a read I/O request from a user-mode process. First, a thread passes the I/O request to the thread's associated environment subsystem (1). The environment subsystem passes this request to the I/O manager (2). The I/O manager interprets the request and builds an IRP (3). Each driver in the stack accesses its I/O stack location, processes the IRP, initializes the stack location for the next driver and passes the IRP to the next driver in the stack (4 and 5). Additionally, a high-level or intermediate driver can divide a single I/O request into smaller requests by creating more IRPs. For example, in the case of a read to a RAID subsystem, the file system driver might construct several IRPs, enabling different parts of the data transfer to proceed in parallel.[303] Any driver, except a low-level driver, can register an **I/O completion routine** with the I/O manager. Low-level drivers complete the I/O request and, therefore, do not need I/O completion routines. When processing of an IRP completes, the I/O manager proceeds up the stack, calling all registered I/O completion. A driver can specify that its completion routine be invoked if an IRP completes successfully, produces an error and/or is canceled. An I/O completion routine can be used to retry an IRP that fails.[304] Also, drivers that create new IRPs are responsible for disposing of them when processing completes; they accomplish task by using an I/O completion routine.[305]

Once the IRP reaches the lowest-level driver, this driver checks that the input parameters are valid, then notifies the I/O manager that an IRP is pending for a particular device (6). The I/O manager determines whether the target device is available.[306] If it is, the I/O manager calls the driver routine that handles the I/O operation (7). The driver routine cooperates with the HAL to direct the device to

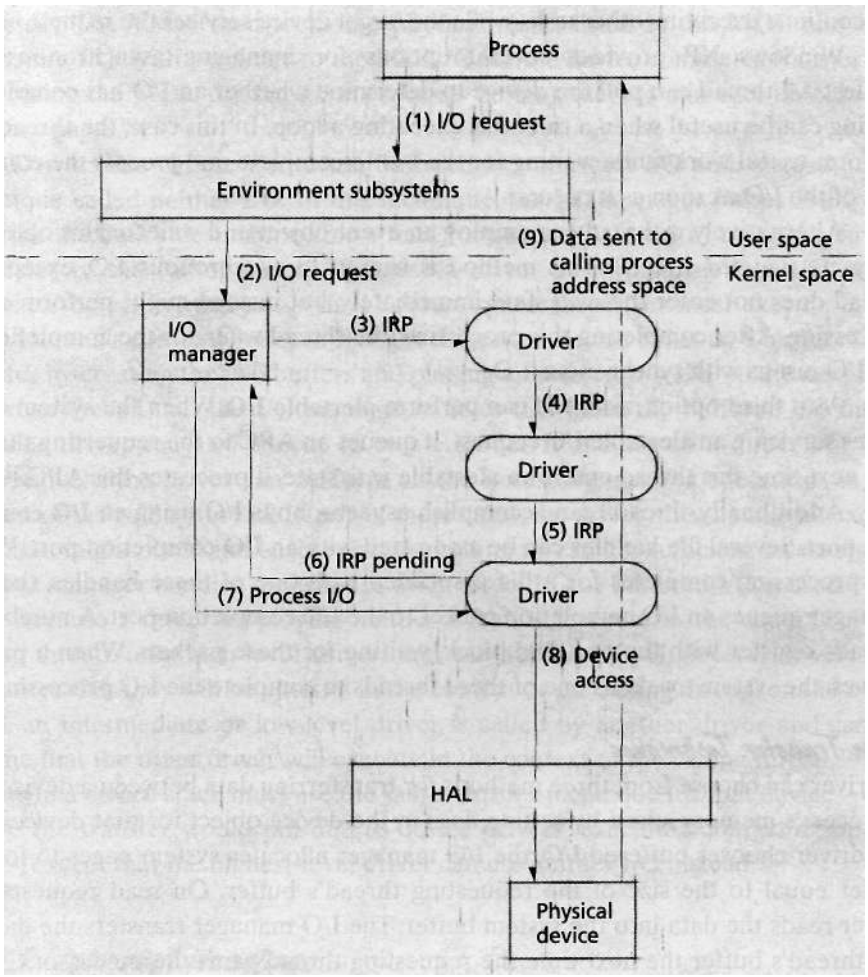


Figure 21.15 | Servicing an IRP.

perform an action on behalf of the requesting thread (8). Finally, the data is sent to the calling process's address space (9). The different methods for transferring I/O are described later in this section. If the device is not available, the I/O manager queues the IRP for later processing.[307] Section 21.9.3, Interrupt Handling, describes how Windows XP completes I/O processing.

Synchronous and Asynchronous I/O

Windows XP supports both synchronous and asynchronous I/O requests. Microsoft uses the term **overlapped I/O** to mean asynchronous I/O. If a thread issues a synchronous I/O request, the thread enters its *wait* state. Once the device services the I/O request, the thread enters its *ready* state, and when it obtains the processor, it

completes I/O processing. However, a thread that issues an asynchronous I/O request can continue executing other tasks while the target device services the request.[308]

Windows XP provides several options for managing asynchronous I/O requests. A thread can poll the device to determine whether an I/O has completed. Polling can be useful when a thread is executing a loop. In this case, the thread can perform useful work while waiting for the I/O to complete and process the completion of the I/O as soon as it occurs.

Alternatively, a thread can employ an event object and wait for this object to enter its *signaled* state.[309] This method is similar to synchronous I/O, except the thread does not enter the *wait* state immediately, but instead might perform some processing. After completing this processing, the thread waits for the completion of the I/O just as with synchronous I/O.

As a third option, a thread can perform **alertable I/O.** When the system completes servicing an alertable I/O request, it queues an APC to the requesting thread. The next time this thread enters an alertable *wait* state, it processes this APC.[310]

Additionally, threads can accomplish asynchronous I/O using an **I/O completion port.** Several file handles can be associated with an I/O completion port. When I/O processing completes for a file associated with one of these handles, the I/O manager queues an I/O completion packet to the I/O completion port. A number of threads register with the port and block, waiting for these packets. When a packet arrives, the system awakens one of these threads to complete the I/O processing.[311]

Data Transfer Techniques

A driver can choose from three methods for transferring data between a device and a process's memory space by setting flags in the device object for that device.[312] If the driver chooses buffered I/O, the I/O manager allocates system pages to form a buffer equal to the size of the requesting thread's buffer. On read requests, the driver reads the data into the system buffer. The I/O manager transfers the data to the thread's buffer the next time the requesting thread gains the processor.[313] For writes, the I/O manager transfers the data from the thread's buffer to the newly allocated system buffer before passing the IRP to the top-level driver.[314] The system buffer does not act as a cache; it is reclaimed after I/O processing completes.

Buffered I/O is useful for small transfers, such as mouse and keyboard input, because the system does not need to lock physical memory pages due to the fact that the system transfers the data first between the device and nonpaged pool. When the VMM locks a memory page, the page cannot be paged out of memory. However, for large transfers (i.e., more than one memory page), the overhead of first copying the data into the system buffer and then making a second copy to the process buffer reduces I/O performance. Also, because system memory becomes fragmented as the system executes, the buffer for a large transfer likely will not be contiguous, further degrading performance.[315]

As an alternative, drivers can employ **direct I/O,** with which the device transfers data directly to the process's buffer. Before passing the IRP to the first driver, the I/O manager creates a **memory descriptor list (MDL),** which maps the applica-

ble range of the requesting process's virtual addresses to physical memory addresses. The VMM then locks the physical pages listed in the MDL and constructs the IRP with a pointer to the MDL. Once the device completes transferring data (using the MDL to access the process's buffer), the I/O manager unlocks the memory pages.[316]

Drivers can choose not to use buffered I/O or direct I/O and, instead, use a technique called **neither I/O.** In this technique, the I/O manager passes IRPs that describe the data's destination (for a read) or origin (for a write) using the virtual addresses of the calling thread. Because a driver performing neither I/O needs access to the calling thread's virtual address space, the driver must execute in the calling thread's context. For each IRP, the driver can decide to set up a buffered I/O transfer by creating the two buffers and passing a buffered I/O IRP to the next lowest driver (recall that the I/O manager handles this when the buffered I/O flag is set). The driver can also choose to create an MDL and set up a direct I/O IRP. Alternatively, the driver can perform all the necessary transfer operations in the context of the calling thread. In any of these cases, the driver must handle all exceptions that might occur and ensure that the calling thread has sufficient access rights. The I/O manager handles these issues when direct I/O or buffered I/O is used.[317]

Because a driver employing neither I/O must execute in the context of the calling thread, only high-level drivers can use neither I/O. High-level drivers can guarantee that they will be entered in the context of the calling thread; on the other hand, an intermediate or low-level driver is called by another driver and cannot assume that the other driver will execute in the context of the calling thread.[318] All drivers in a device stack must use the same transfer technique for that device—otherwise the transfer would fail due to device drivers' executing conflicting operations—except that the highest-level driver can use neither I/O instead.[319]

21.9.3 Interrupt Handling

Once a device completes processing an I/O request, it notifies the system with an interrupt. The interrupt handler calls the **interrupt service routine (ISR)** associated with that device's interrupt.[320] An ISR is a standard driver routine used to process interrupts. It returns false if the device with which it is associated is not interrupting; this can occur because more than one device can interrupt at the same DIRQL. If the associated device is interrupting, the ISR processes the interrupt and returns true.[321] When a device that generates interrupts is installed on a Windows XP system, a driver for that device must register an ISR with the I/O manager. Typically, low-level drivers are responsible for providing ISRs for the devices they serve.

When a driver registers an ISR, the system creates an **interrupt object.**[322] An interrupt object stores interrupt-related information, such as the specific DIRQL at which the interrupt executes, the interrupt vector of the interrupt and the address of the ISR.[323] The I/O manager uses the interrupt object to associate an interrupt vector in the kernel's **Interrupt Dispatch Table (IDT)** with the ISR. The IDT maps hardware interrupts to interrupt vectors.[324]

When an interrupt occurs, the kernel maps a hardware-specific interrupt request into an entry in its IDT to access the correct interrupt vector.[325] Often, there are more devices than entries in the IDT, so the processor must determine which ISR to use. It does this by executing, in succession, each ISR associated with the applicable vector. When an ISR returns false, the processor calls the next ISR.[326]

An ISR for a Windows XP system should execute as quickly as possible, then queue a DPC to finish processing the interrupt. While a processor is executing an ISR, it executes at the device's DIRQL. This masks interrupts from devices with lower DIRQLs. Quickly returning from an ISR, therefore, increases a system's responsiveness to all device interrupts. Also, executing at a DIRQL restricts the support routines that can be called (because some of these routines must run at some IRQL below DIRQL). Additionally, executing an ISR can prevent other processors from executing the ISR—and some other pieces of the driver's code—because the kernel interrupt handler holds the driver's spin lock while processing the **ISR.** Therefore, an ISR should determine if the device with which it is associated is interrupting. If it is not interrupting, the **ISR** should return false immediately; otherwise, it should clear the interrupt, gather the requisite information for interrupt processing, queue a DPC with this information and return true. The driver can use the device extension to store the information needed for the DPC. Typically, the DPC will be processed the next time the **IRQL** drops below DPC/dispatch level.[327]

21.9.4 File Cache Management

Not all I/O operations require the process outlined in the previous sections. When requested data resides in the system's file cache, Windows XP can fulfill an I/O request without a costly device access and without executing device driver code. The **cache manager** is the executive component that manages Windows XP's file cache, which consists of main memory that caches file data.[328] Windows XP does not implement separate caches for each mounted file system, but instead maintains a single systemwide cache.[329]

Windows XP does not reserve a specific portion of RAM to act as the file cache, but instead allows the file cache to grow or shrink dynamically, depending on system needs. If the system is running many I/O-intensive routines, the file cache grows to facilitate fast data transfer. If the system is executing large programs that consume lots of system memory, the file cache shrinks to reduce page faults. Furthermore, the cache manager never knows how much cached data currently resides in physical memory. This is because the cache manager caches by mapping files into the system's virtual address space, rather than managing the cache from physical memory. The memory manager is responsible for paging the data in these views into or out of physical memory. This caching method allows easy integration of Windows XP's two data access methods: memory-mapped files (see Section 13.9, Data Access Techniques) and traditional read/write access. The same file can be accessed via both methods (i.e., as a memory-mapped file through the memory manager and as a traditional file through the file system), because the cache manager maps file data into